Chapter 18 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–17 into pages you can scan in a minute. It has one **cheat sheet** per area, a **"which API for which job"** table, a verified **ECMAScript version table** (ES2015 → ES2026), a **"where to go next"** plan, an **index of the interview questions** in Chapter 17, and an alphabetical **glossary** of 350+ terms, each linked to the section that explains it properly.

Nothing here is new. If a line doesn't make sense, follow its link back to the full explanation. References are written like **§12.2**, meaning Chapter 12, section 2.

What you'll learn: how to use this chapter as a revision tool. Read a sheet, cover the right-hand column, and try to say the answer before you look. The glossary is also a good last check before an interview: if you can't define a term in one sentence, follow its link.

Every concrete value quoted on the sheets (outputs, orderings, return values, errors thrown) is asserted by `code/cheatsheet_checks.mjs`, which runs on Node 24 with jsdom for the DOM sheet:

```
OK   18.1 syntax & types
OK   18.2 operators & coercion
OK   18.3 control flow
OK   18.4 functions/closures/this
OK   18.5 objects/Math/Date/JSON
OK   18.6 classes & prototypes
OK   18.7 strings & regex
OK   18.8 arrays
OK   18.9 Map/Set/iterators
OK   18.10 modules
OK   18.11 errors
OK   18.12 async & event loop
OK   18.13 DOM & events
OK   18.14 browser APIs
OK   18.15 Node core
OK   18.16 testing
      Node v24.11.1 - not yet available: Math.sumPrecise (ES2026), Uint8Array.fromBase64 (ES2026),
Map#getOrInsert (ES2026), Iterator.concat (ES2026), Temporal (ES2027)
OK   18.18 versions (Node 24 availability)
all 17 cheat-sheet checks passed
```

What's inside

#	Sheet	Built from
18.1	Syntax, variables and types	Ch 1, 2
18.2	Operators and the coercion table	Ch 2
18.3	Control flow and loops	Ch 3
18.4	Functions, scope, closures and `this`	Ch 4, 9
18.5	Objects, Math, Date and JSON	Ch 5
18.6	Classes and prototypes	Ch 9
18.7	Strings and regular expressions	Ch 6
18.8	Arrays: mutating vs non-mutating	Ch 7
18.9	Map, Set, iterators and generators	Ch 8
18.10	Modules and tooling	Ch 10
18.11	Errors and debugging	Ch 11
18.12	Async and the event loop (with diagram)	Ch 12
18.13	DOM and events	Ch 13
18.14	Browser APIs and security	Ch 14
18.15	Node.js core modules	Ch 15
18.16	Testing and TypeScript	Ch 16
18.17	Which API for which job	all
18.18	ECMAScript versions ES2015 → ES2026	Ch 1 + TC39
18.19	Where to go next	–
18.20	Interview question index	Ch 17
18.21	Glossary (A–Z)	all

18.1 Syntax, variables and types

Running code (§1.5): the browser console, `<script defer src="app.js">` or `<script type="module">` in a page, `node file.js` / `node file.mjs`, or the Node REPL (`node` with no file).

Basic rules (§1.6): case-sensitive; identifiers use letters, digits, `_` and `$` and can't start with a digit; statements end with `;` (automatic semicolon insertion exists, but never break a line straight after `return`). Comments: `//`, `/* */`, `/** JSDoc */` (§1.7).

Strict mode (§1.8) is on automatically in modules and class bodies, and opt-in with `"use strict"` at the top of a script or function. It turns silent mistakes into errors:

Mistake	Sloppy mode	Strict mode
`x = 1` with no declaration	creates a global	`ReferenceError`
`this` in a plain call `f()`	`globalThis`	`undefined`
Writing a frozen / read-only property	ignored	`TypeError`
`function f(a, a)`, `010`, `with`	allowed	`SyntaxError`

Declaring variables (§2.1, §2.2):

	`const` (default choice)	`let` (when you reassign)	`var` (legacy)
Scope	block `{}`	block `{}`	function
Reassign	✗ `TypeError`	✓	✓
Redeclare in same scope	✗ `SyntaxError`	✗	✓ silently
Read before its line	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)	`undefined`
Top level of a classic script	not on `globalThis`	not on `globalThis`	becomes `globalThis.x`

`const` stops *reassignment*, not *mutation*: `const o = { n: 1 }; o.n = 2` is fine.

The eight types (§2.3): seven primitives plus objects.

Value	`typeof`	Remember
`42`, `3.14`, `NaN`, `Infinity`	`"number"`	one 64-bit float type for all numbers
`10n`	`"bigint"`	whole numbers of any size; can't mix with `number`
`'hi'`	`"string"`	immutable UTF-16 text
`true`	`"boolean"`	
`undefined`	`"undefined"`	"not set yet" — what the language gives you
`null`	`"object"`	a historic bug; test with `x === null`
`Symbol('id')`	`"symbol"`	unique keys
`{}`, `[]`, `new Date()`	`"object"`	arrays too — use `Array.isArray`
`function () {}`, `class {}`	`"function"`	functions are callable objects
an undeclared name	`"undefined"`	`typeof` never throws, except in the TDZ

Primitives vs objects (§2.4): primitives are copied by value and compared by value; objects are copied as a **reference** and compared by identity, so `{} === {}` is `false` and two variables can point at one object.

18.2 Operators and the coercion table

Arithmetic (§2.5):

Expression	Result	Why
`7 / 2`	`3.5`	`/` is always floating point; integer division is `Math.trunc(a / b)`
`-7 % 3`	`-1`	`%` is a remainder that keeps the **left** operand's sign; true modulo is `((a % n) + n) % n` → `2`
`2 ** 10`	`1024`	exponent (ES2016)
`n++` / `++n` (n = 5)	`5` / `7`	postfix returns the old value, prefix the new one
`1 + '2'`	`"12"`	`+` concatenates if either side is a string
`'5' - 2`, `'3' * '4'`	`3`, `12`	other arithmetic converts to numbers
`true + 1`	`2`	`true` → `1`
`[] + []` / `[] + {}` / `[1,2] + [3]`	`""` / `"[object Object]"` / `"1,23"`	objects become strings first

Equality (§2.6). Use `===`. The only common `==` idiom is `x == null` (true for `null` **or** `undefined`).

Expression	==	Rule behind it
`0 == ''`	`true`	string → number (`''` → `0`)
`0 == '0'`	`true`	string → number
`'' == '0'`	`false`	both strings: compared as text — so `==` is **not transitive**
`false == '0'`	`true`	boolean → number first: `0 == '0'`
`'2' == true`	`false`	`true` → `1` , then `2 == 1`
`null == undefined`	`true`	they equal each other…
`null == 0`	`false`	…and nothing else
`null >= 0`	`true`	relational operators turn `null` into `0`
`NaN == NaN`	`false`	`NaN` equals nothing, not even itself
`[1] == 1` , `[] == ![]`	`true` , `true`	object → primitive (`[]` → `''` → `0`)
`1n == 1`	`true`	BigInt vs number compares the maths value
`'10' < '9'` / `'10' < 9`	`true` / `false`	two strings compare by character code; otherwise numerically
`Object.is(NaN, NaN)` / `Object.is(0, -0)`	`true` / `false`	`===` except for those two cases

Conversions (§2.12):

Value	Number(v)	String(v)	Boolean(v)
`''` / `' '`	`0` / `0`	`''` / `' '`	`false` / `true`
`'42'` / `'4px'` / `'0x1f'`	`42` / `NaN` / `31`	same text	`true`
`null`	`0`	`'null'`	`false`
`undefined`	`NaN`	`'undefined'`	`false`
`true` / `false`	`1` / `0`	`'true'` / `'false'`	–
`[]` / `[7]` / `[1,2]`	`0` / `7` / `NaN`	`''` / `'7'` / `'1,2'`	`true` (all objects)
`{}`	`NaN`	`'[object Object]'`	`true`

`parseInt('08.5kg', 10)` → `8` (reads a prefix, **always pass the radix**) but `Number('08.5kg')` → `NaN` (all or nothing).

Falsy (§2.11) — the complete list: `false` , `0` , `-0` , `0n` , `''` , `null` , `undefined` , `NaN` . Everything else is truthy, including `[]` , `{}` , `'0'` , `'false'` and `new Boolean(false)` .

Logical operators (§2.7):

Operator	Returns	Example
`a && b`	`a` if falsy, else `b`	`0 && 'b'` → `0` ; `'a' && 'b'` → `'b'`
`a || b`	`a` if truthy, else `b`	`0 || 'x'` → `'x'`
`a ?? b`	`a` unless `null` / `undefined`	`0 ?? 'x'` → `0` , `'' ?? 'x'` → `''`
`a?.b` , `a?.[k]` , `a?.()`	`undefined` if `a` is `null` / `undefined`	`null?.deep.prop` → `undefined` (short-circuits the whole chain)
`a ||= b` , `a &&= b` , `a ??= b`	assign only when falsy / truthy / nullish	`cfg.port ??= 8080`

Mixing `??` with `||` or `&&` without parentheses is a `SyntaxError`.

Numbers (§2.10): `0.1 + 0.2` is `0.30000000000000004`; integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$ (so `2 ** 53 === 2 ** 53 + 1` is `true`); `1 / 0` is `Infinity`; `Math.max()` is `-Infinity`. Use `Number.isNaN(x)` (the global `isNaN('abc')` converts first and says `true`), `toFixed(2)` for display (`(123.456).toFixed(2)` → `'123.46'`), integer cents for money, `BigInt` beyond 2^{53}. Numeric separators: `1_000_000`. Bitwise operators (§2.8) work on 32-bit integers: `5 & 3` = 1, `5 | 3` = 7, `5 ^ 3` = 6, `~5` = −6.

Precedence in one breath (§2.9): member access/calls → postfix → unary (`!`, `typeof`, `await`) → `**` → `* / %` → `+ -` → shifts → `< > in instanceof` → equality → bitwise → `&&` → `|| ??` → `? :` → assignment and `=>` → comma. When unsure, add parentheses.

18.3 Control flow and loops

Need	Use	Watch out
Choose between code paths	`if / else if / else` (§3.2)	first truthy branch wins – order specific → general
Choose between two values	`cond ? a : b` (§3.3)	don't nest deeply
One value vs many fixed options	`switch` (§3.4)	compares with `===`; forgetting `break` **falls through**; `switch (1) { case 1: … case 2: … break }` runs both cases
Pure value → value mapping	lookup object or `Map`	object keys are strings
Count / index loop	`for (let i = 0; i < n; i++)` (§3.6)	`<` vs `<=` off-by-one
Unknown number of repeats	`while (cond)` (§3.7)	update the condition or it never ends
Run at least once	`do { } while (cond)` (§3.8)	`do {…} while (false)` runs once
Values of an array, string, Map, Set	`for (const v of it)` (§3.10)	plain objects aren't iterable – use `Object.entries(obj)`
Keys of a plain object	`for (const k in obj)` (§3.9)	keys are **strings** (`for…in` over `['a','b']` gives `'0'`, `'1'`), includes inherited ones – avoid on arrays
Index and value	`for (const [i, v] of arr.entries())`	
Leave a loop / skip one pass	`break` / `continue` (§3.11)	`forEach` can't be stopped – use `for…of`, `some`, `find`
Leave an **outer** loop	`outer: for (…) { for (…) { break outer; } }`	labels work with `continue outer` too

Closures in loops (§4.7): `for (var i…)` shares one `i`, so three saved callbacks all return `3`; `for (let i…)` makes a fresh binding per iteration, giving `0, 1, 2`.

18.4 Functions, scope, closures and `this`

Forms (§4.1–§4.3):

```
function add(a, b) { return a + b; }        // declaration: hoisted with its body
const sub = function (a, b) { return a - b; }; // expression: follows const (TDZ)
const sq = (x) => x * x;                    // arrow, implicit return
const mk = (x, y) => ({ x, y });            // returning an object literal needs ( )
(() => { /* IIFE: runs immediately */ })();
```

	Regular `function`	Arrow `=>`
`this`	set by the call site	lexical — taken from where it's written; `call` / `bind` can't change it
`arguments`	yes	no — use `...args`
`new`	allowed	`TypeError` (no `prototype`)
Best as	object methods, constructors	callbacks, especially inside methods

Parameters (§4.4): with `f = (a, b = 2, ...rest) => [a, b, rest]`, `f(1)` → `[1, 2, []]`; `f(1, undefined, 3, 4)` → `[1, 2, [3, 4]]`; `f(1, null)` → `[1, null, []]` — **defaults replace only `undefined`**. `fn.length` counts parameters before the first default/rest (`(a, b = 1, c) => 0` has length `1`). Use an options object for named parameters: `function f({ a = 1, b = 2 } = {})`.

Scope and hoisting (§4.5, §4.6):

Declaration	Before its line
`function f() {}`	callable
`var x`	`undefined`
`let` / `const` / `class`	`ReferenceError` (TDZ)
`var f = function…`	`undefined` → calling it is a `TypeError`

Lookups walk the **lexical scope chain** outward from where the code is *written*; an inner name shadows an outer one.

Closures (§4.7): a function keeps live access to the variables of the scope it was created in.

```
function counter() { let c = 0; return () => ++c; }
const c1 = counter(); c1(); c1();
const c2 = counter();
c1(); // 3   - c1 has its own c
c2(); // 1   - c2 has a separate c
```

Uses: private state, factories, `once`, `memoize`, event handlers, debounce/throttle (§17.3).

Higher-order functions (§4.8): take or return functions — `map/filter/reduce`, `pipe` (left → right, `reduce`), `compose` (right → left, `reduceRight`), currying `f(a)(b)(c)`, partial application. **Recursion** (§4.10): base case + smaller case; too deep → `RangeError: Maximum call stack size exceeded`. **Pure functions** (§4.11): same input → same output, no side effects.

`this` — decided at call time (§9.1), highest priority first:

Call site	`this`
`new Fn()`	the new object
`fn.call(obj, a, b)`, `fn.apply(obj, [a, b])`, `fn.bind(obj)()`	`obj`
`obj.fn()`	`obj` (the thing before the dot)
plain `fn()`	`undefined` in strict code (`globalThis` in sloppy scripts)
arrow function	whatever `this` was where the arrow was written

Checked facts: `user.hi()` → `'Ada'`, but `const lost = user.hi; lost()` → `this` is `undefined` (the method "loses" its object when passed around — fix with an arrow wrapper or `bind`). Binding twice keeps the **first** binding. `call` runs now with arguments listed, `apply` runs now with an array, `bind` returns a new function (§9.2).

`['1', '7', '11'].map(parseInt)` → `[1, NaN, 3]`, because `map` passes `(value, index)` and `parseInt` treats the index as the radix.

18.5 Objects, Math, Date and JSON

Objects (§5.1–§5.5):

```
const key = 'dyn';
const o = { b: 1, 2: 'two', a: 2, 1: 'one', [key]: 3 };   // computed key
Object.keys(o);            // ['1', '2', 'b', 'a', 'dyn'] - integer-like keys first, then insertion order
const { a, b: renamed, missing = 'def' } = o;            // 2, 1, 'def'
const merged = { ...{ x: 1, y: 1 }, ...{ y: 2 } };       // { x: 1, y: 2 } - later wins
```

Task	Code	Note
Read with a fixed / dynamic key	`o.name` / `o[expr]`	brackets for spaces, dashes, variables
Own key?	`Object.hasOwn(o, k)`	`'toString' in {}` is `true` (inherited); `Object.hasOwn({}, 'toString')` is `false`
Remove	`delete o.k`	
List	`Object.keys / values / entries`, rebuild with `Object.fromEntries`	own, enumerable, string keys
Shallow copy / merge	`{ ...a, ...b }`, `Object.assign({}, a, b)`	nested objects are **shared**
Deep copy	`structuredClone(o)`	handles Date, Map, Set, cycles; throws on functions
Lock	`Object.preventExtensions` < `Object.seal` < `Object.freeze`	all **shallow**: a frozen object's nested objects stay writable
Prototype	`Object.create(proto)`, `Object.create(null)` for a bare dictionary	§18.6
Getter / setter	`get total() {…}`, `set total(v) {…}` (§5.7)	keep them cheap

Descriptors (§5.6): `writable`, `enumerable`, `configurable` (+ `get` / `set`). Normal assignment makes them all `true`; `Object.defineProperty` defaults them to `false`.

Math (§5.10):

Call	Result
`Math.round(2.5)` / `Math.round(-2.5)`	`3` / `-2` (halves go toward +∞)
`Math.floor(-1.5)` / `Math.trunc(-1.5)` / `Math.ceil(1.1)`	`-2` / `-1` / `2`
`Math.max(...[3, 9, 1])`	`9` (spread — `max` takes separate arguments)
Random int in [min, max]	`Math.floor(Math.random() * (max - min + 1)) + min`
Secure random	`crypto.getRandomValues`, `crypto.randomUUID()` — never `Math.random`

Date (§5.11): a `Date` is milliseconds since 1970-01-01 UTC. Months are **0-based** (`new Date(2024, 0, 31)` is 31 January); `getDay()` is the weekday (0 = Sunday), `getDate()` the day of month; setters mutate and roll over (setting month 1 on 31 January gives a date in March); a date-only ISO string like `'2024-03-10'` is parsed as **UTC midnight**; `new Date(0).toISOString()` → `'1970-01-01T00:00:00.000Z'`. Show dates with `Intl.DateTimeFormat` / `toLocaleDateString`. **Temporal** reached Stage 4 in March 2026 (it goes into ES2027); it is not in Node 24, so feature-detect it.

JSON (§5.12):

Input to `JSON.stringify`	Output
`{ u: undefined, f() {}, n: NaN, d: new Date(0), m: new Map([[1, 2]]) }`	`{"n":null,"d":"1970-01-01T00:00:00.000Z","m":{}}`
`[undefined, () => 1]`	`[null,null]`
`undefined` (top level)	`undefined` (not a string)
`{ big: 1n }` or an object with a cycle	throws `TypeError`
`JSON.stringify(v, null, 2)`	pretty-printed with 2-space indent

`JSON.parse` is strict: double-quoted keys and strings only, no trailing commas, no comments (`JSON.parse("{'a':1}")` throws `SyntaxError`). A **reviver** `(key, value) => …` can turn ISO strings back into `Date`s.

18.6 Classes and prototypes

```
class Animal {
  #secret = 's';                    // private field (ES2022)
  static count = 0;                 // static field
  static { Animal.ready = true; }   // static block, runs once
  constructor(name) { this.name = name; Animal.count++; }
  speak() { return `${this.name} makes a sound`; }   // on Animal.prototype
  get upper() { return this.name.toUpperCase(); }     // accessor
  static isAnimal(x) { return #secret in x; }        // brand check
}
class Dog extends Animal {
  speak() { return `${super.speak()} - woof`; }
}
new Dog('Rex').speak();  // 'Rex makes a sound - woof'
```

Fact	Checked result
`Object.keys(new Dog('Rex'))`	`['name']` – methods are non-enumerable, `#private` is invisible
`Object.getPrototypeOf(Dog.prototype) === Animal.prototype`	`true` – `extends` links the chains
`Animal('x')` without `new`	`TypeError`
Using `this` before `super()` in a subclass constructor	`ReferenceError`
`typeof class {}`	`'function'` – a class is a special function
`Object.getPrototypeOf(Object.prototype)`	`null` – the end of every chain
`'toString' in Object.create(null)`	`false` – no inherited keys

How lookup works (§9.3): reading a missing property walks the `[[Prototype]]` chain until it finds it or reaches `null`; writing always creates an **own** property (shadowing). `Fn.prototype` is the object that *instances*

of `new Fn()` get as their prototype, not `Fn`'s own prototype. `__proto__` is a legacy accessor — use `Object.getPrototypeOf` / `Object.create`.

`new Fn(args)` (§9.4): 1. create an object linked to `Fn.prototype`; 2. run `Fn` with `this` = that object; 3. return it, unless `Fn` returns a different **object** (a returned primitive is ignored).

Type checks (§9.7): `typeof` for primitives, `Array.isArray` for arrays, `Error.isError` (ES2026) for errors, `x instanceof C` for "made by C" (walks the chain; breaks across realms), `#brand in x` inside a class, duck typing (`typeof x?.then === 'function'`) for "behaves like", `Object.prototype.toString.call(x)` → `'[object Date]'` for built-in tags.

Design (§9.8, §9.9): inheritance (is-a) only for stable taxonomies such as `HttpError extends Error`; composition (has-a) for most application code; mixins to share a capability between unrelated classes. TypeScript's `private` is compile-time only; JavaScript's `#private` is enforced at runtime.

18.7 Strings and regular expressions

Strings (§6.2–§6.6) are immutable UTF-16 primitives. Methods always return a **new** string.

Job	Method	Example → result
Interpolate / multi-line	template literal	`` `${1 + 1}` `` → `'2'`
Last char, out-of-range	`at(-1)`, `s[i]`, `charAt(i)`	`'Hello'.at(-1)` → `'o'`; `'Hello'[10]` → `undefined`; `charAt(10)` → `''`
Substring	`slice(start, end?)` (negatives OK)	`'Hello'.slice(-3)` → `'llo'`, `slice(1, 3)` → `'el'`
Search	`includes`, `startsWith`, `endsWith`, `indexOf` (–1 if absent)	
Replace first / all	`replace` / `replaceAll`	`'a-b-c'.replace('-', '+')` → `'a+b-c'`; `replaceAll` → `'a+b+c'`
Pad, repeat	`padStart`, `padEnd`, `repeat`	`'5'.padStart(3, '0')` → `'005'`
Trim	`trim`, `trimStart`, `trimEnd`	
Split / join	`split(sep)`, `arr.join(sep)`	`'a,b,,c'.split(',')` → `['a', 'b', '', 'c']`
Case	`toUpperCase`, `toLowerCase`, `toLocale…`	
Human ordering	`a.localeCompare(b)`, `Intl.Collator`	default `sort` gives `'Cab'` for `['b','a','C']`; with `localeCompare` → `'abC'`

Unicode (§6.5): `length` counts 16-bit code units — `'😀'.length` is `2`, `[...'😀'].length` is `1` (code points), and a family emoji `'👨‍👩‍👧'` is one grapheme to `Intl.Segmenter`. Normalize with `normalize('NFC')` before comparing accented text.

Regex syntax (§6.7):

Piece	Meaning
`.` `\d` `\w` `\s` (`\D` `\W` `\S`)	any char (not newline) / digit / word char / whitespace (and complements)
`[abc]` `[^abc]` `[a-z]`	class / negated class / range
`*` `+` `?` `{n,m}`	0+ / 1+ / 0–1 / n to m — **greedy**; add `?` for lazy
`^` `$` `\b`	start / end / word boundary
`( )` `(?<name> )` `(?: )`	capture / named capture / group only
`(?= )` `(?! )` `(?<= )` `(?<! )`	lookahead / negative lookahead / lookbehind / negative lookbehind
`\1`, `\k<name>`	backreference
`\p{L}` (with `u` or `v`)	any letter in any script
Flags	`g` all, `i` ignore case, `m` multiline anchors, `s` dot matches newline, `u` Unicode, `v` Unicode sets, `y` sticky, `d` indices

Regex methods (§6.8):

Call	Result
`/^\d{3}-\d{4}$/.test('555-1234')`	`true`
`'x'.match(/y/)`	`null` (not `[]`)
`'a1b22'.match(/\d+/g)`	`['1', '22']`
`'2024-03-10'.match(/(?<y>\d{4})-…/).groups.y`	`'2024'`
`'2024-03-10'.replace(/(\d+)-(\d+)-(\d+)/, '$3/$2/$1')`	`'10/03/2024'`
`[...'a1b2'.matchAll(/\d/g)].map(m => m.index)`	`[1, 3]`; `matchAll` without `g` throws `TypeError`
`'<b>x</b>'.replace(/<.+>/, '')` vs `/<.+?>/`	`''` (greedy) vs `'x</b>'` (lazy)
`/a/g.test('aa')` then `lastIndex`	`1` — a `g` regex remembers its position between calls
`new RegExp(RegExp.escape(userText))`	matches the text literally (ES2025)

Prefer plain string methods for fixed text and real parsers (`URL`, `JSON.parse`, `DOMParser`) for structured formats (§6.9).

18.8 Arrays: mutating vs non-mutating

✎ = changes the original array (§7.4).

Mutates ✎	Returns	Copying twin / alternative (leaves the original alone)
`push(x)` / `pop()`	new length / removed item	`[...arr, x]` / `arr.slice(0, -1)`
`unshift(x)` / `shift()`	new length / removed item	`[x, ...arr]` / `arr.slice(1)`
`splice(i, n, ...items)`	array of **removed** items	`toSpliced(i, n, ...items)` (ES2023)
`sort(cmp)`	the **same** array	`toSorted(cmp)` (ES2023)
`reverse()`	the same array	`toReversed()` (ES2023)
`arr[i] = x`	–	`with(i, x)` (ES2023; negative `i` OK, out of range → `RangeError`)
`fill(v, s, e)`, `copyWithin(t, s, e)`	the same array	`Array.from` / `map`
`arr.length = n`	truncates	`slice(0, n)`

Never mutated: `slice`, `concat`, `map`, `filter`, `reduce`, `flat`, `flatMap`, `find*`, `some`, `every`, `includes`, `indexOf`, `at`, `join`, `keys/values/entries`, spread.

Checked behaviours: `const s = arr.sort(); s === arr` → `true`; `[1].push(2)` → `2` (not the array); `[10, 1, 2].sort()` → `[1, 10, 2]` (default sort compares **strings**) but `.sort((a, b) => a - b)` → `[1, 2, 10]`; `[3, 1, 2].toSorted()` → `[1, 2, 3]` with the original unchanged.

Iteration methods (§7.5):

Want	Method	Checked example
Transform each	`map`	
Keep some	`filter`	
Fold to one value	`reduce(fn, init)`	`[1,2,3].reduce((s, x) => s + x, 0)` → `6`; `[].reduce(fn)` with no init → `TypeError`
First match / its index	`find` / `findIndex`	`find(x => x > 1)` → `2`; `findIndex(x => x > 5)` → `-1`
Last match	`findLast` / `findLastIndex` (ES2023)	`findLast(x => x > 1)` → `3`
Any / all	`some` / `every`	`[].every(…)` → `true` (vacuous truth)
Contains	`includes`	`[NaN].includes(NaN)` → `true`, `[NaN].indexOf(NaN)` → `-1`
Flatten	`flat(depth = 1)`, `flatMap`	`[1,[2,[3,[4]]]].flat()` → `[1,2,[3,[4]]]`; `flat(Infinity)` → `[1,2,3,4]`
Side effects only	`forEach` / `for…of`	`forEach` can't `break` or `await`

Sorting (§7.6): numbers `(a, b) => a - b`; text `a.localeCompare(b)`; several keys chained with `||`: `(p, q) => p.age - q.age || p.name.localeCompare(q.name)`. `sort` is stable (guaranteed since ES2019).

Creating and destructuring (§7.2, §7.7): `Array.from({ length: 3 }, (_, i) => i * 2)` → `[0, 2, 4]`; `Array.of(3)` → `[3]` but `new Array(3)` has length 3 and holes; `Array(2).fill([])` puts the **same** inner array in both slots. `[x, y] = [y, x]` swaps; `const [first, , third = 'd', ...others] = [1, 2, undefined, 4, 5]` → `1`, `'d'`, `[4, 5]`. `join` turns `null` / `undefined` into empty strings.

Immutable updates (React/Redux style): add `[...arr, x]`; insert `arr.toSpliced(i, 0, x)`; remove by id `arr.filter(t => t.id !== id)`; replace `arr.with(i, x)`; update one `arr.map(t => t.id === id ? { ...t, done: true } : t)`.

Which collection (§7.8): array for ordered lists, object for fixed-shape records, `Map` for dynamic keyed lookup, `Set` for uniqueness. **Typed arrays** (§7.9) hold fixed-size binary numbers over an `ArrayBuffer`; values wrap (`new Uint8Array([256, 257])` → `[0, 1]`).

18.9 Map, Set, iterators and generators

`Map` (§8.1) – keys of any type, insertion order, `size`:

```
const m = new Map([[1, 'num'], ['1', 'str']]);   // 1 and '1' are different keys
m.set({}, 'obj').set(NaN, 'nan');                 // set() returns the map, so it chains
m.get(NaN);            // 'nan' - keys compare with SameValueZero
JSON.stringify(m);     // '{}'  - convert first: Object.fromEntries(m) or [...m]
for (const [k, v] of m) { /* insertion order */ }
```

`Set` (§8.2) – unique values, O(1) `has`: `[...new Set([3, 1, 3, 2, 1])]` → `[3, 1, 2]` (dedupe, first-seen order). ES2025 set algebra, with A = {1,2,3} and B = {2,3,4}:

Method	Result
`A.union(B)`	{1, 2, 3, 4}
`A.intersection(B)`	{2, 3}
`A.difference(B)`	{1}
`A.symmetricDifference(B)`	{1, 4}
`A.isSubsetOf(B)`, `isSupersetOf`, `isDisjointFrom`	booleans

Use	When
`Map`	dynamic dictionaries, counting, caches (an LRU cache is one `Map`), non-string keys
plain object	fixed-shape records, config, JSON
`Set`	membership tests, dedupe, visited sets in BFS/DFS
`WeakMap` / `WeakSet` (§8.3)	data attached to objects you don't own; entries vanish with the key object; no `size`, no iteration
`WeakRef` / `FinalizationRegistry`	advanced caches; GC timing is never guaranteed

Iteration protocol (§8.4): an **iterable** has a `[Symbol.iterator]()` method returning an **iterator**, an object whose `next()` returns `{ value, done }`. Arrays, strings, Maps, Sets, typed arrays, NodeLists and generators are iterable; plain objects are not (`[...{ a: 1 }]` → `TypeError`). Consumers: `for…of`, spread, array destructuring, `Array.from`, `new Map/Set`, `Promise.all`, `yield*`.

Generators (§8.5):

```
function* nat() { let n = 1; while (true) yield n++; }   // infinite, lazy
nat().next();                          // { value: 1, done: false }
const iterable = { *[Symbol.iterator]() { yield 1; yield 2; } };
[...iterable];                         // [1, 2]
function* two() { yield 'a'; return 'r'; }
[...two()];                            // ['a'] - the return value is not iterated
```

Iterator helpers (ES2025, §8.6) are lazy: `nat().filter(n => n % 2).map(n => n * n).take(3).toArray()` → `[1, 9, 25]`. Arrays need `.values()` first; `Iterator.from(x)` wraps custom iterators.

Grouping (ES2024, §8.7): `Object.groupBy([1,2,3,4], n => n % 2 ? 'odd' : 'even')` → `{ odd: [1, 3], even: [2, 4] }` (a null-prototype object); `Map.groupBy` when keys aren't strings.

Async iteration (§12.12): `async function*` + `for await (const page of pages())`; `await Array.fromAsync(pages())` (ES2026) collects them.

18.10 Modules and tooling

```
// math.mjs
export const PI = 3.14;                 // named export
export function area(r) { return PI * r * r; }
export default class Circle {}          // one default per module
// app.mjs
import Circle, { area, PI as pi } from './math.mjs';
import * as math from './math.mjs';     // namespace object: math.area, math.default
export { area } from './math.mjs';      // re-export (barrel file)
const { heavy } = await import('./heavy.mjs');  // dynamic import -> promise, code splitting
import data from './data.json' with { type: 'json' };  // JSON module (ES2025)
```

ES module fact (§10.3)	Consequence
Loads in 3 phases: parse → link → evaluate	imports are hoisted; a missing export fails before any code runs
Imports are **live, read-only bindings**	you see the exporter's updates; you can't assign to them
Each module is evaluated **once** (singleton)	module-level state is shared by every importer
Always strict; top-level `this` is `undefined`	checked by the sheet script
Top-level `await` allowed (§10.5)	importers wait for it
`import.meta.url`, `import.meta.dirname` / `filename` (Node) (§10.6)	replace CommonJS `__dirname` / `__filename`

CommonJS vs ESM in Node (§10.8): `.mjs` = ESM, `.cjs` = CommonJS, `.js` follows `"type"` in the nearest `package.json`. CommonJS uses `require()` / `module.exports`, loads synchronously, has `__dirname`. ESM can import CommonJS (default import = `module.exports`); since Node 22.12 / 20.19, CommonJS can `require()` an ES module unless it uses top-level `await` (`ERR_REQUIRE_ASYNC_MODULE`). **Browsers** (§10.7): `<script type="module">` is deferred, strict, needs CORS and a server (not `file://`); import maps resolve bare names.

npm (§10.9, §10.10):

Item	Meaning
`dependencies` / `devDependencies` / `peerDependencies`	needed at runtime / only to build and test / "use the host's copy"
`^1.2.3`	≥1.2.3 <2.0.0 (npm's default)
`~1.2.3`	≥1.2.3 <1.3.0
`^0.2.3`	≥0.2.3 <0.3.0 (caret is stricter below 1.0)
`package-lock.json`	commit it; CI runs `npm ci` (exact, clean install)
`npx tool`, `npm run x`, `node --run x`	run a package binary / a script

Tools (§10.11–§10.13): bundlers (Vite, esbuild, webpack, Rollup/Rolldown) do resolution, tree shaking, code splitting, minification, source maps and HMR. **Transpile syntax, polyfill APIs**: a transpiler rewrites new syntax for old engines; a polyfill adds missing built-ins. ESLint finds bugs (flat config `eslint.config.js`); Prettier formats.

18.11 Errors and debugging

Error type (§11.1)	Checked trigger
`TypeError`	`null.length`, calling a non-function, `1n + 1`, assigning to a `const`
`ReferenceError`	undeclared name, TDZ access
`SyntaxError`	`JSON.parse('{bad}')`, bad code in `eval` / `new Function`
`RangeError`	`new Array(-1)`, `(1).toFixed(101)`, stack overflow
`URIError`	`decodeURIComponent('%')`
`AggregateError`	`Promise.any` when all reject – `.errors` holds them

```
class ValidationError extends Error {                // §11.3
  constructor(msg, opts) { super(msg, opts); this.name = 'ValidationError'; this.code = 'E_VALID'; }
}
try {
  save();
} catch (err) {
  if (!(err instanceof ValidationError)) throw err;   // catch narrowly, rethrow the rest
  throw new Error('save failed', { cause: err });     // §11.4 error chaining
} finally {
  release();                                          // always runs; never return from it
}
```

Rules: always throw `Error` objects (they carry a `stack`); `String(err)` → `'ValidationError: save failed'`; a `return` in `try` still runs `finally` first; `catch {}` without a binding is legal (ES2019) but never swallow errors silently. **`try/catch` is synchronous**: it catches a rejected `await` inside an `async` function, but not an error thrown later in a timer callback (§11.5). Global last-resort handlers: browser `error` / `unhandledrejection` events; Node `uncaughtException` / `unhandledRejection` – log, then exit. `using` + `Symbol.dispose` gives automatic cleanup (§11.6; finished proposal, edition ES2027).

Debugging (§11.7–§11.9): `console.table`, `dir`, `group`, `count`, `time` / `timeEnd`, `trace`, `assert` (logs, doesn't throw). Stack traces list the newest frame first; source maps map bundled code back to your files. Breakpoints (conditional, logpoints, pause on exceptions), step over / into / out, Scope and Watch panes; `debugger;` statement; Node: `node --inspect` then `chrome://inspect`. Method: reproduce → read the message → hypothesize → inspect → fix the root cause → add a test. Common messages decoded: §11.10.

18.12 Async and the event loop

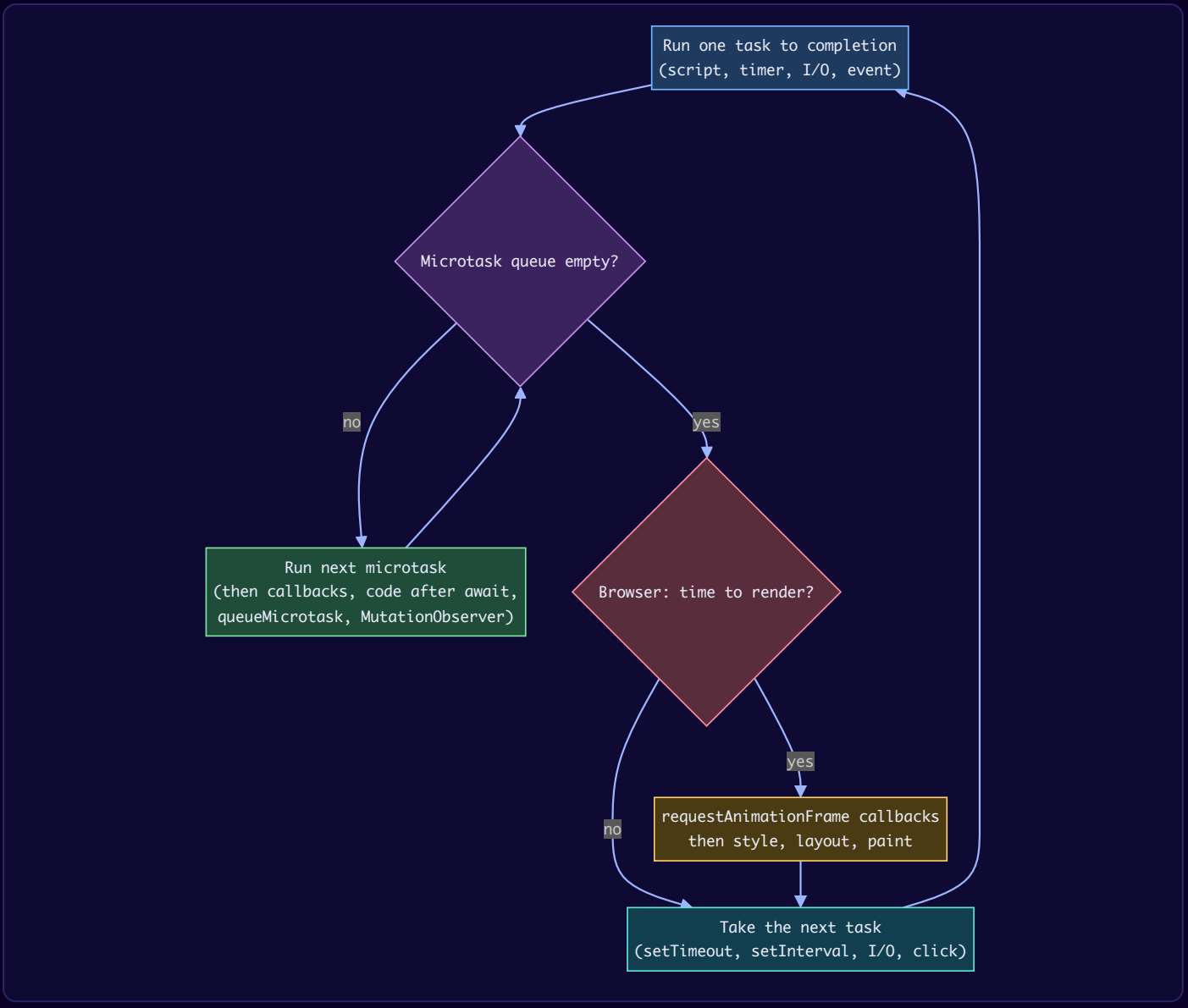


*Figure — one turn of the event loop: one task, then **all** microtasks, then maybe a render (§12.2).*

The ordering rule (§12.3): synchronous code first (including promise executors and an async function's body up to its first `await`), then microtasks in FIFO order, then the next task.

```
log('A');
setTimeout(() => log('B timeout'), 0);
Promise.resolve().then(() => log('C then')).then(() => log('D then2'));
queueMicrotask(() => log('M micro'));
new Promise((res) => { log('P executor'); res(); });
(async () => { log('F before await'); await null; log('F after await'); })();
log('E');
// A, P executor, F before await, E, C then, M micro, F after await, D then2, B timeout
```

Node extras (§12.4): phases timers → pending → poll (I/O) → check (`setImmediate`) → close. Inside a callback, `process.nextTick` runs before promise microtasks, and both run before `setImmediate` — checked order: `nextTick, promise, immediate`.

Promises (§12.7, §12.8): pending → fulfilled or rejected, settle **once**; `then` returns a new promise, so always `return` inside it and end the chain with `catch`.

Combinator	Fulfils when	Rejects when	Checked
`Promise.all`	all fulfil – values in **input order**	first rejection	`[ok(1, 5ms), ok(2, 1ms)]` → `[1, 2]`
`Promise.allSettled`	all settle	never	statuses `['fulfilled', 'rejected']`
`Promise.race`	first to **settle** fulfils	first to settle rejects	fastest wins
`Promise.any`	first to **fulfil**	all reject → `AggregateError`	skips a rejection
`Promise.withResolvers()` (ES2024)	`{ promise, resolve, reject }`		
`Promise.try(fn)` (ES2025)	a sync throw becomes a rejection		

None of the combinators cancel the losing operations.

async/await (§12.9): an `async` function always returns a promise; `await` unwraps it and **throws** on rejection (so `try/catch` works). Two independent awaits one after another take the sum of their times (checked: ~80 ms); `await Promise.all([...])` takes the longest (~40 ms). `arr.forEach(async …)` doesn't wait – use `for…of` (one at a time) or `Promise.all(arr.map(async …))` (all at once). Use `return await` inside `try` so the `catch` sees the rejection.

Timers (§12.5): delays are minimums; nested timers clamp to 4 ms in browsers; delays above $2^{31} - 1$ ms overflow; prefer recursive `setTimeout` to `setInterval` for polling. **Cancellation** (§12.11): `const ac = new AbortController(); fetch(url, { signal: ac.signal }); ac.abort()`; `AbortSignal.timeout(ms)`; `AbortSignal.any([...])`. **Patterns** (§12.13): retry with exponential backoff + jitter, concurrency limits, last-request-wins, debounce/throttle.

18.13 DOM and events

Task	API (§13.1–§13.4)
Find one / many	`document.getElementById('id')`, `el.querySelector(css)` / `querySelectorAll(css)` (static `NodeList`)
Live collections	`getElementsByTagName` / `ByClassName` – update automatically (checked: after appending an `<li>`, the live list has 2 items, the static one still 1)
Relatives	`el.closest(css)` (self or ancestor), `el.matches(css)`, `parent.contains(node)`, `children`, `nextElementSibling`
Create and insert	`document.createElement('li')`, `append`, `prepend`, `before`, `after`, `replaceWith`, `remove`, `replaceChildren`; `DocumentFragment` / `<template>` for bulk
Text vs HTML	`el.textContent = userText` (never parsed – `'<b>safe</b>'` shows as text); `innerHTML` only for trusted markup (XSS risk)
Classes, data, style	`classList.add/remove/toggle/contains`, `el.dataset.id` (from `data-id`, always a string), `el.style.x` (inline), `getComputedStyle(el)` (final)
Attribute vs property	`input.getAttribute('value')` = initial HTML value; `input.value` = current value

Events (§13.5–§13.8):

```
list.addEventListener('click', (e) => {          // one listener for all items (delegation)
  const item = e.target.closest('li');           // target = where it started
  if (!item || !list.contains(item)) return;
  console.log(item.dataset.id, e.currentTarget); // currentTarget = list (the listener's element)
}, { signal: controller.signal });                // options: once, passive, capture, signal
```

Fact	Checked
Propagation: capture (window → target), target, bubble (target → window)	order: `body capture`, `delegated 7`, `body bubble target=BUTTON current=BODY`
`new Event('ping')` doesn't bubble	parent heard 0; with `{ bubbles: true }` → 1
`{ once: true }`	handler ran once for two clicks
`e.preventDefault()`	cancels the default action (link navigation, form submit); `e.defaultPrevented` → `true`
`stopPropagation()`	stops the journey – use rarely
`dispatchEvent`	runs listeners **synchronously**

Forms (§13.9): listen for `submit`, call `preventDefault()`, read `new FormData(form)` (`getAll('t')` → `['a', 'b']` for repeated names), check `input.validity` / `setCustomValidity`, use `form.requestSubmit()`; always validate again on the server. **Watch changes**: `MutationObserver` (§13.10). **Performance** (§13.11): batch all layout reads, then all writes (interleaving them causes **layout thrashing**); animate `transform` / `opacity`; use `requestAnimationFrame`. **Script timing** (§13.12): `defer` and modules run after parsing, in order, before `DOMContentLoaded`; `async` runs whenever downloaded; `load` waits for images and styles.

18.14 Browser APIs and security

fetch (§14.1):

```
async function getJSON(url, { timeout = 5000 } = {}) {
  const res = await fetch(url, { signal: AbortSignal.timeout(timeout),
                                 headers: { Accept: 'application/json' } });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch resolves on 404/500!
  return res.json();                                    // the body can be read only once
}
await fetch('/api/items', { method: 'POST',
  headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(item) });
```

`fetch` rejects only on network failure or abort; a 404 `Response` has `ok === false`. Reading a body twice throws `TypeError`.

URL (§14.6): `new URL('https://shop.example.com:8443/search?q=red shoes')` → `.origin` = `'https://shop.example.com:8443'`, `.searchParams.get('q')` = `'red shoes'`; after `set('page','3')` and `append('tag','a&b')` the search string is `'?q=red+shoes&page=3&tag=a%26b'` (encoding done for you). `new URL('../img/a.png', 'https://x.dev/app/page/')` resolves to `https://x.dev/app/img/a.png`; `new URL('not a url')` throws `TypeError` (a cheap validator). `encodeURIComponent('a b&c')` → `'a%20b%26c'`.

Storage (§14.3, §14.4):

	Cookie	localStorage	sessionStorage	IndexedDB
Size	~4 KB	~5 MB	~5 MB	large
API	strings, sync	strings, sync (`setItem('n', 42)` stores `'42'` ; missing → `null`)	same, per tab	async, structured data
Sent to server	automatically	no	no	no
Use for	session ID (`HttpOnly; Secure; SameSite`)	preferences	wizard state	offline data, big caches

Never put tokens or secrets in `localStorage` – any script on the page can read it.

Other APIs: `requestAnimationFrame` for visual updates, once per frame, paused in background tabs (§14.5); History API `pushState` / `replaceState` + `popstate` for SPA routing (§14.7); Web Workers for CPU work off the main thread, talking via `postMessage` + structured clone (§14.8); `IntersectionObserver` (visibility, lazy loading) and `ResizeObserver` (element size) (§14.9). `structuredClone` copies Dates and Maps but throws `DataCloneError` on functions.

Security (§14.2, §14.10–§14.12):

Threat / rule	Remember
Origin	scheme + host + port; the browser lets a page *send* anywhere but *read* only same-origin responses
CORS	the **server** opts in with `Access-Control-Allow-Origin` ; non-simple requests get an `OPTIONS` preflight; `*` can't be combined with credentials
XSS	treat data as text (`textContent`), escape by context (`<` → `<`), sanitize HTML with DOMPurify, allow-list URL schemes, CSP, `HttpOnly` cookies
CSRF	`SameSite` cookies + CSRF token or `Origin` check; GET must never change state; CORS is not a CSRF defence
CSP	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'` – blocks inline scripts and clickjacking
Other headers	HSTS, `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy`
Supply chain	lockfiles, `npm audit` , few dependencies, Subresource Integrity for CDN scripts

---

18.15 Node.js core modules

Node = V8 + **libuv** (event loop, async I/O, a 4-thread pool for fs, DNS lookup, crypto, zlib) + core modules (§15.1, §15.2). Import built-ins with the `node:` prefix. Never block the loop; move CPU-heavy work to `worker_threads` .

Module	Everyday calls	Checked / remember
`node:fs/promises` (§15.4)	`readFile(p, 'utf8')`, `writeFile`, `appendFile`, `mkdir(p, { recursive: true })`, `readdir`, `stat`, `rm(p, { recursive: true, force: true })`	branch on `err.code` (`'ENOENT'`); don't check-then-act
`node:path` (§15.5)	`join`, `resolve`, `basename`, `extname`, `dirname`, `parse`, `relative`	`join('/srv','app','..','data')` → `/srv/data`; `extname('price.test.mjs')` → `.mjs`
`node:url`	`fileURLToPath(import.meta.url)`, `pathToFileURL`	or `import.meta.dirname`
`process` (§15.6)	`argv`, `env`, `exitCode`, `cwd()`, `on('SIGTERM')`	`process.env.X = 5` reads back `'5'` – env values are strings; `--env-file=.env`
`node:util`	`parseArgs`, `promisify`, `inspect`	`parseArgs` on `['--port','8080','file.txt']` → `port: '8080'`, positionals `['file.txt']`
`Buffer` (§15.7)	`Buffer.from(str)`, `.toString('hex' / 'base64')`	`Buffer.from('é').length` → `2` bytes, `'é'.length` → `1`
`node:events` (§15.8)	`on`, `once`, `off`, `emit`; `events.once(em, name)` → promise	`emit` is **synchronous**; an unhandled `'error'` event throws
`node:stream` (§15.9)	Readable, Writable, Duplex, Transform; `pipeline` from `node:stream/promises`	`pipeline` handles **backpressure** and errors
`node:http` (§15.10)	`createServer((req, res) => { … res.end() })`, `listen(0)` in tests	always end the response; limit body size
`node:child_process` (§15.11)	`execFile(cmd, args)`, `spawn`	pass argument arrays – no shell injection
`node:test`, `node:assert/strict`	§18.16	

Flags (§15.12): `--watch`, `--env-file`, `--run`, `--test`, `--inspect`, `--permission`; `node app.ts` strips types (no type check). **Alternatives**: Deno (secure by default, V8) and Bun (JavaScriptCore, fast tooling); Node stays the default choice.

18.16 Testing and TypeScript

```
// price.test.mjs - run with: node --test
import { test, describe, beforeEach } from 'node:test';
import assert from 'node:assert/strict';
import { applyDiscount } from './price.mjs';

describe('applyDiscount', () => {
  test('takes 10 percent off', () => assert.equal(applyDiscount(100, 10), 90));
  test('rejects negative percentages', () => assert.throws(() => applyDiscount(100, -1), RangeError));
  test('async loader rejects', async () => { await assert.rejects(load('bad'), /not found/); });
  test('calls the logger once', (t) => {
    const log = t.mock.fn();                    // spy, auto-restored
    applyDiscount(100, 10, { log });
    assert.equal(log.mock.callCount(), 1);
  });
});
```

Assertion (§16.2)	Passes when	Checked
`equal(a, b)`	`Object.is(a, b)` — same primitive or same object	`equal({a:1}, {a:1})` fails
`deepEqual(a, b)`	same structure, type-strict	`{a:1}` vs `{a:'1'}` fails
`throws(fn, Class | /regex/)`	`fn()` throws a match	
`await rejects(promise, …)`	the promise rejects — **must be awaited**	
`match(str, /re/)`	regex matches	

`node:test` (§16.3–§16.6): `test` / `it`, `describe`, hooks `before` / `beforeEach` / `afterEach` / `after`, `skip` / `todo` / `only`; `node --test` (finds `*.test.*`), `--watch`, `--test-name-pattern`; `t.mock.fn` / `t.mock.method` spies; fake timers `t.mock.timers.enable()` + `tick(ms)`; snapshots; `--experimental-test-coverage`. Test pyramid: many unit, some integration, few end-to-end (Playwright). Vitest/Jest use `expect(x).toBe(y)` (§16.7).

TypeScript (§16.8–§16.13):

```
type Shape = { kind: 'circle'; r: number } | { kind: 'rect'; w: number; h: number }; // discriminated union
function area(s: Shape): number {
  switch (s.kind) {                              // narrowing by discriminant
    case 'circle': return Math.PI * s.r ** 2;
    case 'rect': return s.w * s.h;
    default: { const never: never = s; return never; }   // exhaustive check
  }
}
function first<T>(xs: readonly T[]): T | undefined { return xs[0]; }   // generic
const ROLES = ['admin', 'user'] as const;            // readonly literal tuple
type Role = (typeof ROLES)[number];                  // 'admin' | 'user'
```

Concept	One line
Types are erased	nothing is checked at runtime — validate JSON with a schema (e.g. Zod)
Structural typing	shapes match, names don't matter
`unknown` vs `any`	`unknown` must be narrowed before use; `any` switches checking off
Narrowing	`typeof`, `instanceof`, `in`, discriminant property, type predicates `x is T`, `asserts` functions
Utility types	`Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `Awaited`, `NonNullable`
`interface` vs `type`	both describe objects; `type` also does unions/tuples; interfaces merge
`satisfies`	checks a value against a type without widening it
Running	`tsc --noEmit` type-checks; `node file.ts` only strips erasable syntax (no `enum`, no parameter properties)
JSDoc	`// @ts-check` + `/** @param {number} x */` type-checks plain `.js` files (§16.15)

18.17 Which API for which job

Job	Reach for	Not	See
Default variable	`const` (`let` if reassigned)	`var`	§2.1
Compare values	`===`, `Object.is` for `NaN` / `-0`	`==` (except `x == null`)	§2.6
Default for a missing value	`??` / `??=`	`||` when `0` or `''` are valid	§2.7
Read possibly-missing nested data	`a?.b?.c`	long `&&` chains	§5.8
Money	integer cents, `Intl.NumberFormat` for display	floats	§2.10
Integers beyond 2^{53}	`BigInt`	`number`	§2.10
Parse user input to a number	`Number(s)` + `Number.isNaN`	`parseInt` without a radix	§2.12
Walk an array / string / Map	`for…of`	`for…in`	§3.12
Walk a plain object	`Object.entries(obj)` + `for…of`	`for…in` without `hasOwn`	§5.6
Transform / select / fold	`map` / `filter` / `reduce`	`forEach` + push	§7.5
Sort without touching the original	`toSorted(cmp)`	`sort()` on shared data	§7.4
Last element	`arr.at(-1)`	`arr[arr.length - 1]`	§7.3
Copy an object / array (top level)	`{ ...o }`, `[...a]`	`Object.assign` into the original	§5.4
Deep copy	`structuredClone`	`JSON.parse(JSON.stringify(x))`	§5.4
Dictionary with dynamic or non-string keys	`Map`	plain object	§8.1
Unique values, fast membership	`Set`	`array.includes` in a loop	§8.2
Metadata on objects you don't own	`WeakMap`	`Map` (leaks)	§8.3
Group items by a key	`Object.groupBy` / `Map.groupBy`	hand-written `reduce`	§8.7
Lazy or infinite sequence	generator + iterator helpers	building a huge array	§8.5
Text search / replace of fixed text	`includes`, `replaceAll`	regex	§6.6
Pattern matching	regex (`test`, `matchAll`, named groups)	manual char loops	§6.8
Count visible characters	`Intl.Segmenter`	`length`	§6.5
Format dates / numbers for people	`Intl.DateTimeFormat`, `toLocaleString`	string concatenation	§5.11
Callback that keeps `this`	arrow function	`const self = this`	§9.1
Truly private state	`#private` fields or closures	`_underscore` names	§9.5

Job	Reach for	Not	See
Share code between modules	ES modules (`import` / `export`)	globals, IIFEs	§10.2
Load code only when needed	`import()`	one giant bundle	§10.4
Reproducible installs	lockfile + `npm ci`	`npm install` in CI	§10.10
Report a failure	`throw new Error(msg, { cause })` / custom subclass	throwing strings	§11.3
Several independent async jobs	`await Promise.all([...])`	sequential awaits	§12.9
Report every outcome	`Promise.allSettled`	`all`	§12.8
Timeout or cancel	`AbortSignal.timeout(ms)` , `AbortController`	`Promise.race` alone (doesn't cancel)	§12.11
Wrap an old callback API	`util.promisify` or `new Promise` once	nesting callbacks	§12.7
Paged / streamed data	`async function*` + `for await…of`	loading everything first	§12.12
Insert user text into the page	`textContent`	`innerHTML`	§13.3
Handle events on many children	delegation with `closest`	a listener per item	§13.7
Remove many listeners at once	`{ signal }` + `controller.abort()`	tracking every function	§13.5
Animate	CSS transitions, or `requestAnimationFrame` changing `transform`	`setInterval` changing `top` / `left`	§14.5
React to scroll visibility / element size	`IntersectionObserver` / `ResizeObserver`	`scroll` / `resize` handlers with measurements	§14.9
HTTP request	`fetch` + check `res.ok`	`XMLHttpRequest`	§14.1
Build or parse URLs	`URL` , `URLSearchParams`	string concatenation	§14.6
Store a session	`HttpOnly; Secure; SameSite` cookie	`localStorage` token	§14.4
Large offline data	IndexedDB	`localStorage`	§14.4
Heavy computation in a page	Web Worker	a long loop on the main thread	§14.8
Heavy computation in Node	`worker_threads`	blocking the event loop	§15.2
Big files / HTTP bodies	streams + `pipeline`	`readFile` of gigabytes	§15.9
Run another program	`execFile` / `spawn` with an argument array	`exec` with user input	§15.11
Tests without dependencies	`node:test` + `node:assert/strict`	ad-hoc `console.log` checks	§16.3
Time-dependent tests	fake timers (`t.mock.timers`)	real `setTimeout` waits	§16.5
Types for untrusted data	`unknown` + narrowing / a schema	`any`	§16.10
A fixed set of names in TS	union of literals or `as const` object	`enum` (not erasable)	§16.13

18.18 ECMAScript versions ES2015 → ES2026

Each edition is published in June and contains every proposal that reached **Stage 4** in the year before (§1.2). The years below were checked against TC39's list of finished proposals (github.com/tc39/proposals, "finished-proposals"), and the Node 24 column was checked by running the features in Node v24.11 (the `18.18` block of `cheatsheet_checks.mjs`).

Edition	Main features	Node 24
ES2015 (ES6)	`let` / `const`, arrow functions, classes, ES modules, promises, template literals, destructuring, default / rest / spread parameters, `Map` / `Set` / `WeakMap` / `WeakSet`, `Symbol`, iterators and generators, `for…of`, `Proxy` / `Reflect`, typed arrays, `Array.from` / `of`, `find` / `findIndex`, `Object.assign`, `String` `includes` / `startsWith` / `repeat`, `Number.isNaN` / `isInteger`, regex `u` and `y` flags	✓
ES2016	`Array.prototype.includes`, exponent operator `**`	✓
ES2017	`async` / `await`, `Object.values` / `entries`, `padStart` / `padEnd`, `Object.getOwnPropertyDescriptors`, trailing commas in parameter lists, `SharedArrayBuffer` and `Atomics`	✓
ES2018	object rest/spread `{ ...o }`, async iteration (`for await…of`, async generators), `Promise.prototype.finally`, regex `s` flag, named groups, lookbehind, `\p{…}` escapes	✓
ES2019	`flat` / `flatMap`, `Object.fromEntries`, `trimStart` / `trimEnd`, optional `catch` binding, `Symbol.prototype.description`, well-formed `JSON.stringify`; `sort` required to be stable	✓
ES2020	`BigInt`, `??`, `?.`, dynamic `import()`, `import.meta`, `globalThis`, `Promise.allSettled`, `String.prototype.matchAll`, defined `for…in` order	✓
ES2021	`replaceAll`, `Promise.any` + `AggregateError`, logical assignment `||=` `&&=` `??=`, numeric separators `1_000`, `WeakRef` / `FinalizationRegistry`	✓
ES2022	class fields, `#private` methods and fields, `static` blocks, `#x in obj`, top-level `await`, `.at()`, `Object.hasOwn`, error `cause`, regex `d` flag (match indices)	✓
ES2023	`toSorted` / `toReversed` / `toSpliced` / `with`, `findLast` / `findLastIndex`, hashbang `#!` lines, symbols as `WeakMap` keys	✓
ES2024	`Object.groupBy` / `Map.groupBy`, `Promise.withResolvers`, regex `v` flag, `isWellFormed` / `toWellFormed`, resizable `ArrayBuffer`, `ArrayBuffer.prototype.transfer`, `Atomics.waitAsync`	✓
ES2025	iterator helpers + `Iterator.from`, `Set` methods (`union`, `intersection`, …), `Promise.try`, `RegExp.escape`, regex pattern modifiers `(?i:…)`, duplicate named groups, import attributes + JSON modules, `Float16Array` / `Math.f16round`	✓
ES2026	`Array.fromAsync`, `Error.isError`, `JSON.parse` source text access (`JSON.rawJSON`), `Math.sumPrecise`, `Uint8Array` to/from base64 and hex, `Iterator.concat`, upsert (`Map.prototype.getOrInsert` / `getOrInsertComputed`)	partial: first three ✓; the rest ✗ (feature-detect)
Next: ES2027	finished but not yet published: **Temporal**, explicit resource management (`using`, `Symbol.dispose`), `Atomics.pause`, joint iteration (`Iterator.zip`)	`using` and `Atomics.pause` already work; Temporal and `Iterator.zip` don't

"Node 24 ✓" means Node's V8 ships the feature. For browsers, check MDN's compatibility tables or the Baseline badge before relying on anything from the last two editions. A transpiler can rewrite new *syntax*, but only a

polyfill can add missing *APIs* (§10.12).

18.19 Where to go next

The course behind this book closes by inviting feedback and pointing to further courses. A list of places is more useful with an order, so here is a four-step study path built on what this book covered.

Step 1 — Lock in the core (1–2 weeks)

1. Re-read the TL;DR at the end of each chapter, then do each sheet in this chapter with the right-hand column covered.
2. Work through the ordering puzzles in §12.3 and the 53 questions in §17.4 until you can explain each answer out loud.
3. Rebuild from memory: `debounce`, `throttle`, `curry`, `memoize`, a `Promise.all` polyfill and an LRU cache (§17.3).

Step 2 — Build small, real things (2–4 weeks)

Pick projects that force you to combine chapters:

Project	Exercises
To-do list with filters, saved in `localStorage`	DOM, events, delegation, immutable array updates, storage (Ch 13, Ch 14)
Search-as-you-type against a public API	`fetch`, `AbortController`, debounce, last-request-wins, error states (Ch 12)
A command-line tool that summarizes a CSV or log file	`node:fs/promises`, streams, `parseArgs`, exit codes (Ch 15)
A tiny JSON REST API with tests	`node:http`, validation, `node:test`, TypeScript via type stripping (Ch 15, Ch 16)

Write tests for each one, run ESLint and Prettier, and put it in a git repository.

Step 3 — Choose a direction

Direction	Next topics	In this library
Front end	semantic HTML, CSS layout (flexbox, grid), accessibility, then React (components, state, effects, routing, data fetching)	HTML & CSS Handbook, React Handbook
Back end with Node	a framework (Express, Fastify or Hono), SQL and schema design, authentication, caching, queues	SQL Handbook, Database modeling
Types at scale	strict TypeScript, generics, declaration files, typing React components	§16.8, TypeScript with React
Interviews	DSA patterns in JS, machine-coding rounds, object-oriented and system design	Ch 17, LLD Handbook, DDIA notes

Step 4 — Keep up to date

- **MDN Web Docs** is the reference for every API in this book, with browser-compatibility tables.
- **The ECMAScript spec** (tc39.es/ecma262) and the TC39 proposals repository show what's coming; features at Stage 3 are usually safe to learn.
- **Node.js release notes and docs** (nodejs.org): use an even-numbered LTS line.
- **Books for depth**: *You Don't Know JS Yet* (scope, closures, `this`, types), *Eloquent JavaScript* (fundamentals with exercises), and javascript.info as a free, well-structured tutorial.
- Read other people's code: small, well-maintained npm packages are a good place to start.

18.20 Interview question index

All questions below are answered, with real output, in Chapter 17. Links go to the group that contains the question.

Coercion and equality → §17.4.1

Q	Topic	Q	Topic
1	`+` with strings, arrays and objects	8	falsy values, `[]` and `new Boolean(false)`
2	`null == undefined`, `null >= 0`, `'' == 0`	9	`'b' + 'a' + +'a' + 'a'`
3	why `[] == ![]` is true	10	`['1','7','11'].map(parseInt)`
4	`typeof null`	11	`[10, 1, 2].sort()`
5	how `NaN` behaves	12	`||` vs `??`
6	`Object.is` vs `===`	13	`{} === {}`
7	`typeof` for arrays, functions, classes, undeclared names	14	`0.1 + 0.2` and `Math.max() < Math.min()`

Scope, hoisting and closures → §17.4.2: Q15 `var` vs `let` vs `const` · Q16 reading before the declaration (TDZ) · Q17 calling a function before its definition · Q18 closures in loops · Q19 what a closure is · Q20 `let a = b = 5` · Q21 `var` leaking out of a block · Q22 what strict mode changes.

`this`, arrows, `call` / `apply` / `bind`, `new` → §17.4.3: Q23 a method losing `this` · Q24 arrow vs regular functions · Q25 nested function vs nested arrow · Q26 `call` vs `apply` vs `bind` · Q27 binding twice · Q28 what `new` does and constructor return values.

Prototypes and classes → §17.4.4: Q29 `prototype` vs `__proto__` · Q30 how `instanceof` works · Q31 classes vs constructor functions · Q32 `Object.create(null)` · Q33 `#private` vs closure privacy · Q34 `this` before `super()`.

Event loop and async → §17.4.5: Q35 the classic timer/promise order · Q36 `async` / `await` order · Q37 `nextTick`, promises, `setImmediate` · Q38 executor timing and resolving twice · Q39 why `try/catch` misses a timer's error · Q40 `await` inside `forEach` · Q41 two independent awaits · Q42 `all` vs `allSettled` vs `race` vs `any`.

Objects, arrays and copies → §17.4.6: Q43 shallow vs deep copy · Q44 changing a `const` object, `Object.freeze` · Q45 objects as object keys · Q46 `delete arr[i]` and `arr.length = n` · Q47 pass-by-value or by reference · Q48 `Map` vs object · Q49 what `JSON.stringify` drops · Q50 `for…in` vs `for…of`.

Memory → §17.4.7: Q51 how memory is freed · Q52 causes of leaks and fixes · Q53 `WeakMap`, `WeakRef`, `FinalizationRegistry`.

Also in Chapter 17: the complexity table of built-ins (§17.1), twenty-one coding patterns with a summary table (§17.2), and implement-from-scratch classics – debounce, throttle, curry, compose/pipe, memoize, flatten, `bind`, deep clone, deep equal, LRU cache, EventEmitter, promise combinator polyfills (§17.3).

Coding patterns at a glance → §17.2

#	Pattern	Reach for it when…	Classic problems in §17.2
1–3	Two pointers, sliding window, hashing	sorted pairs; contiguous ranges; "seen before"	pair sum, longest unique substring, two sum, anagram groups
4–6	Stack, prefix sums, intervals	nesting / next greater; range sums; overlaps	brackets, daily temperatures, subarray sum = k, merge, meeting rooms
7	Heap, two heaps, k-way merge	top k, cheapest next, running median	k largest, Dijkstra, median of a stream, merge k sorted lists
8–10	Binary search, BFS, DFS	sorted / monotone yes-no; fewest steps; connectivity	lower bound, eating speed, grid path, islands, DFS topo order
11–12	DP with memo, backtracking	overlapping subproblems; list every option	stairs, coin change, LCS, subsets, N-Queens
13	Linked lists	rewire `next` references	reverse, remove k-th from end, intersection
14	Fast & slow pointers	cycle or middle in O(1) space	cycle start, middle node, happy number
15	Trees	combine left/right subtree answers	validate BST, lowest common ancestor, serialize
16	Tries	prefix lookups, many words at once	prefix tree, wildcard match, words on a board
17	Topological sort, union-find	dependencies; merging groups	course order (Kahn), components, redundant edge
18	Greedy	a local choice you can prove safe	jump game, gas station, candies
19	Sort & search	k-th item, 3-way partition, stable sort	quickselect, Dutch flag, merge sort
20	Bit manipulation	xor cancelling, bit counts, masks (32-bit!)	single number, counting bits, swap odd/even bits
21	Math & geometry	matrix walks, integer limits, exact slopes	spiral order, reverse integer, max points on a line

---

18.21 Glossary (A–Z)

Each term has a one-line definition and a link to the section that explains it. Symbols and backticks are ignored when sorting.

A

- **AbortController** – An object whose `signal` is passed to `fetch`, listeners or timers; calling `abort()` cancels them all. §12.11
- **AbortSignal.timeout** – A ready-made signal that aborts itself after a number of milliseconds with a `TimeoutError`. §12.11
- **Accessor property** – A property backed by a getter and/or setter function instead of a stored value. §5.7
- `addEventListener` – Registers a function to run when an event of a given type reaches an element; accepts `once`, `passive`, `capture` and `signal` options. §13.5
- **AggregateError** – An error that wraps several errors in its `errors` array; `Promise.any` rejects with one when every input rejects. §11.1
- **AMD / UMD** – Older module formats: AMD loads modules asynchronously with `define()`; UMD wraps a file so it works as AMD, CommonJS or a global. §10.1
- **Anchor (regex)** – A position rather than a character: `^` start, `$` end, `\b` word boundary. §6.7
- `arguments` – An array-like object holding every argument passed to a non-arrow function; replaced in modern code by rest parameters. §4.4
- **Array destructuring** – Unpacking values by position: `const [a, , c = 0, ...rest] = arr`. §7.7

- **Array-like** — An object with `length` and numeric indexes but no array methods (e.g. `arguments`, a `NodeList`); convert with `Array.from`. §4.4
- **`Array.from`** — Builds a real array from an iterable or array-like, with an optional mapping function. §7.2
- **`Array.fromAsync`** — Collects an async iterable (or iterable of promises) into an array, returning a promise (ES2026). §12.12
- **`Array.isArray`** — The reliable array test; works across realms, unlike `instanceof Array`. §7.1
- **ArrayBuffer** — A fixed block of raw bytes that typed arrays and `DataView` read and write. §7.9
- **Arrow function** — Short function syntax `(x) => x * 2` with lexical `this`, no `arguments` and no `new`. §4.3
- **Assertion** — A check in a test that fails loudly when a value isn't what you expected (`assert.equal`, `assert.throws`). §16.2
- **Assertion function (TS)** — A TypeScript function declared `asserts v is T` that narrows its argument for the code after the call. §16.11
- **Assignment operators** — `=` plus compound forms such as `+=`, `**=`, `||=`, `&&=` and `??=`. §2.8
- **`async` function** — A function that always returns a promise and may use `await` inside. §12.9
- **Async generator** — An `async function*` that yields values over time; consumed with `for await…of`. §12.12
- **Async iteration** — Walking a sequence whose items arrive asynchronously, using `Symbol.asyncIterator` and `for await…of`. §12.12
- **Attribute (HTML)** — The text written in markup (`value="init"`); read with `getAttribute`. Often the initial value only. §13.4
- **Auto-boxing** — The engine briefly wraps a primitive in its object wrapper so that `'abc'.toUpperCase()` works. §6.4
- **`await`** — Pauses an async function until a promise settles; gives the value or throws the rejection reason. §12.9

B

- **Babel** — A transpiler that rewrites modern JavaScript (and JSX/TypeScript) into syntax older engines understand. §10.12
- **Backpressure** — A stream's signal that the consumer is slower than the producer; `write()` returns `false` until `'drain'`. §15.9
- **Backreference** — `\1` or `\k<name>` in a regex: match the same text a group matched earlier. §6.7
- **Barrel file** — A module that only re-exports other modules (`export * from './a.mjs'`) to offer one import path. §10.2
- **Base case** — The condition that stops a recursive function from calling itself again. §4.10
- **BigInt** — A primitive for integers of any size, written `10n`; can't be mixed with `number` in arithmetic. §2.10
- **`bind`** — Returns a new function with `this` (and optionally leading arguments) fixed permanently. §9.2
- **Bitwise operators** — `& | ^ ~ << >> >>>`: operate on the 32-bit integer form of numbers. §2.8
- **Block scope** — Visibility limited to the nearest `{ }`; applies to `let`, `const` and `class`. §4.5
- **Bubbling** — The phase in which an event travels from the target back up through its ancestors. §13.6
- **Buffer** — Node's type for raw bytes; its length is in bytes, not characters. §15.7
- **Bun** — A JavaScript runtime built on JavaScriptCore with its own package manager, bundler and test runner. §15.12
- **Bundler** — A tool that follows imports and produces optimized files for the browser (Vite, esbuild, webpack, Rollup). §10.11

C

- **`call` / `apply`** — Invoke a function right now with a chosen `this`; `call` takes listed arguments, `apply` an array. §9.2
- **Call stack** — The stack of function calls currently running; the newest frame is on top. §12.2
- **Callback** — A function passed to another function to be called later or for each item. §4.8
- **Callback hell** — Deeply nested callbacks where each step depends on the previous one; hard to read and to handle errors. §12.6
- **Capture phase** — The first event phase, travelling from `window` down to the target; listen with `{ capture: true }`. §13.6
- **Capturing group** — `( … )` in a regex: groups a sub-pattern and remembers its match as `$1`, `m[1]`. §6.7

- `cause` (error) — Option `new Error(msg, { cause })` that links an error to the one that caused it (ES2022). §11.4
- Character class — `[abc]`, `[a-z]` or shorthands like `\d`: matches one character from a set. §6.7
- Child process — Another OS program started from Node with `node:child_process` (`execFile`, `spawn`, `fork`). §15.11
- Class — Syntax for a constructor plus prototype methods, with fields, `#private` members, statics and `extends`. §9.5
- Class field — A property declared in the class body (`count = 0`) and created on each instance (ES2022). §9.5
- `classList` — An element's list of CSS classes with `add`, `remove`, `toggle` and `contains`. §13.4
- Closure — A function together with the variables of the scope where it was created, which it can still read and change. §4.7
- Code point — A Unicode character number (U+0000 to U+10FFFF); some need two UTF-16 code units. §6.5
- Code splitting — Breaking a bundle into chunks that load on demand, usually at each dynamic `import()`. §10.11
- Code unit — One 16-bit value of UTF-16; `length` and indexes count these. §6.5
- Coercion — Automatic type conversion done by operators such as `+`, `==` and `if`. §2.12
- CommonJS — Node's original module system: `require()` and `module.exports`, loaded synchronously. §10.8
- Comparator — The function passed to `sort`: negative keeps `a` first, positive puts `b` first, 0 keeps their order. §7.6
- Composition — Building behaviour by combining objects or functions (has-a) instead of inheriting (is-a). §9.9
- Computed property name — An object key written as an expression in brackets: `{ [key]: value }`. §5.2
- `const` — Declares a block-scoped binding that can't be reassigned (the value may still be mutated). §2.1
- Constructor function — A regular function called with `new` to create objects; the pre-class way to make "types". §9.4
- Content Security Policy (CSP) — A response header that tells the browser which sources of scripts, styles and frames are allowed. §14.12
- Cookie — A small string the server sets and the browser sends back automatically with matching requests. §14.4
- CORS — Cross-Origin Resource Sharing: headers by which a server allows pages from other origins to read its responses. §14.2
- Coverage — The share of code lines or branches your tests executed; a hint, not a goal. §16.6
- CSRF — Cross-site request forgery: another site makes the victim's browser send an authenticated request. §14.11
- `currentTarget` — The element whose listener is running now, as opposed to `target`, where the event started. §13.5
- Currying — Turning `f(a, b, c)` into `f(a)(b)(c)` so arguments can be supplied one at a time. §4.8
- CustomEvent — An event you create with your own `detail` data and fire with `dispatchEvent`. §13.5

D

- `dataset` — Access to `data-*` attributes as camelCase string properties (`data-user-id` → `dataset.userId`). §13.4
- `Date` — A built-in object holding milliseconds since 1970-01-01 UTC, with local and UTC getters. §5.11
- Debounce — Run a function only after calls have stopped for a given time (e.g. search-as-you-type). §17.3
- `debugger` — A statement that pauses execution when developer tools are attached. §11.9
- Deep copy — A copy in which nested objects are copied too, e.g. `structuredClone(obj)`. §5.4
- Default export — The single unnamed export of a module, imported under any name: `import x from './m.mjs'`. §10.2
- Default parameter — A value used when an argument is missing or `undefined`: `function f(n = 1)`. §4.4
- `defer` — Script attribute: download in parallel, run after the HTML is parsed, in document order. §13.12
- `delete` — Operator that removes an own property from an object. §5.2
- Deno — A V8-based runtime with deny-by-default permissions, built-in TypeScript and tooling. §15.12

- **Dependency injection** – Passing collaborators (clock, database, HTTP client) in as arguments so tests can swap them for fakes. §16.5
- **Destructuring** – Syntax that unpacks properties or elements into variables, with renames and defaults. §5.5
- `devDependencies` – Packages needed only to build and test (linters, bundlers), not at runtime. §10.9
- **Discriminated union** – A TypeScript union whose members share a literal "kind" property used to narrow between them. §16.11
- `dispatchEvent` – Fires an event object at a node; its listeners run synchronously. §13.5
- **DocumentFragment** – A lightweight container for building many nodes off-page before inserting them at once. §13.3
- **DOM** – Document Object Model: the live tree of objects the browser builds from HTML and exposes to scripts. §13.1
- `DOMContentLoaded` – Event fired when the HTML is parsed and deferred scripts have run; images may still be loading. §13.12
- **DOMPurify** – A widely used library that sanitizes untrusted HTML before it goes into `innerHTML`. §14.10
- **dotAll flag** – The regex `s` flag: lets `.` match newline characters too. §6.7
- **Duck typing** – Checking whether a value has the methods you need rather than which class made it. §9.7
- **Dynamic import** – `import(specifier)`: loads a module at runtime and returns a promise for its namespace. §10.4
- **Dynamic typing** – Types belong to values, not variables; a variable may hold a number and later a string. §1.1

E

- **ECMAScript** – The official language standard (ECMA-262) that JavaScript engines implement; a new edition every June. §1.2
- **End-to-end (E2E) test** – A test that drives the whole application through its real interface, often a browser. §16.1
- **Engine** – The program that parses, compiles and runs JavaScript: V8, SpiderMonkey, JavaScriptCore. §1.3
- **Enumerable** – A property attribute; enumerable properties appear in `for…in`, `Object.keys` and spread. §5.6
- **Error-first callback** – Node's callback convention `(err, result) => …`, where `err` is checked first. §12.6
- `Error.isError` – Reliable check that a value is a genuine error object, even across realms (ES2026). §9.7
- **ES module (ESM)** – The standard module system: static `import` / `export`, strict mode, live bindings. §10.2
- **esbuild** – A very fast bundler and transpiler written in Go. §10.11
- **ESLint** – A configurable linter that reports likely bugs and style problems in JavaScript and TypeScript. §10.13
- **Event** – An object describing something that happened (click, key press, load) delivered to listeners. §13.5
- **Event delegation** – One listener on a container handles events from many children using `event.target.closest()`. §13.7
- **Event loop** – The host scheduler that runs one task, then all microtasks, then possibly a render, repeatedly. §12.2
- **EventEmitter** – Node's publish/subscribe class: `on`, `once`, `off` and synchronous `emit`. §15.8
- **Exponentiation operator** – `**`: raises to a power (`2 ** 10` is 1024). §2.5
- `export` – Makes a binding available to other modules, by name or as the default. §10.2
- **Expression** – Code that produces a value (`2 + 3`, `f(x)`, `a ? b : c`). §1.6

F

- **Fake timers** – Test tool that replaces `setTimeout` / `Date` so you can move time forward instantly. §16.5
- **Falsy** – A value that becomes `false` in a boolean context: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. §2.11
- `fetch` – The promise-based HTTP API in browsers and Node; resolves even for 4xx/5xx responses. §14.1
- `finally` – Block that runs after `try` / `catch` whatever happened, for cleanup. §11.2
- **First-class function** – Functions are values: stored in variables, passed as arguments and returned from other functions. §4.1
- `flat` / `flatMap` – Flatten nested arrays by a depth / map then flatten one level. §7.5
- `for await…of` – Loops over an async iterable, awaiting each value. §12.12

- `for…in` – Loops over enumerable property names (strings), including inherited ones. §3.9
- `for…of` – Loops over the values of any iterable (arrays, strings, Maps, Sets, generators). §3.10
- **FormData** – An object holding a form's field names and values; can be sent directly with `fetch`. §13.9
- **Function declaration** – `function name() {}` as a statement; hoisted with its body. §4.2
- **Function expression** – A function used as a value (`const f = function () {}`); follows its variable's hoisting. §4.2

G

- **Garbage collection** – Automatic freeing of memory for objects no longer reachable from roots such as globals and the stack. §17.4.7
- **Generator** – A `function*` that can pause at `yield`; calling it returns an iterator. §8.5
- **Generics (TS)** – Type parameters such as `<T>` that let one function or type work for many types safely. §16.12
- **Getter** – A `get` method that runs when a property is read. §5.7
- `globalThis` – The standard name for the global object in every environment (ES2020). §1.4
- **Grapheme cluster** – What a reader sees as one character, possibly several code points (flags, family emoji). §6.5
- **Greedy quantifier** – A regex repetition that matches as much as possible; add `?` to make it lazy. §6.7
- `groupBy` – `Object.groupBy` / `Map.groupBy` put items into buckets by a key function (ES2024). §8.7

H

- `hasOwn` – `Object.hasOwn(obj, key)`: is this an own (not inherited) property? (ES2022) §5.2
- **Higher-order function** – A function that takes or returns another function. §4.8
- **History API** – `pushState`, `replaceState` and `popstate` for changing the URL without reloading the page. §14.7
- **HMR** – Hot Module Replacement: a dev server swaps changed modules into the running page without a full reload. §10.11
- **Hoisting** – Declarations are registered before code runs: functions fully, `var` as `undefined`, `let` / `const` / `class` in the TDZ. §4.6
- **Host** – The environment around the engine that supplies extra APIs: a browser, Node.js, Deno, Bun. §1.4
- `HttpOnly` – Cookie attribute that hides the cookie from JavaScript, protecting sessions from XSS theft. §14.4

I

- **IEEE 754** – The 64-bit floating-point format behind every JavaScript `number`. §2.10
- **IIFE** – Immediately Invoked Function Expression `(() => { … })()`: an old way to get a private scope. §4.9
- **Immutability** – Not changing data in place; producing new values instead (important in React and Redux). §7.4
- **Implicit return** – An arrow function with an expression body returns that expression without `return`. §4.3
- `import` – Brings exported bindings from another module into this one. §10.2
- **Import attributes** – `with { type: 'json' }` on an import, telling the host how to load the module (ES2025). §10.6
- **Import map** – A browser `<script type="importmap">` that maps bare specifiers like `'lodash'` to URLs. §10.7
- `import.meta` – Per-module metadata such as `url`, `dirname`, `filename` and `resolve()`. §10.6
- **IndexedDB** – The browser's asynchronous, transactional database for large structured data. §14.4
- **Inheritance** – One class extending another (`extends`) and reusing its behaviour through the prototype chain. §9.6
- `innerHTML` – Gets or sets an element's children as an HTML string; setting it parses markup (XSS risk). §13.3
- `instanceof` – Tests whether `C.prototype` appears in an object's prototype chain. §9.7
- **Integration test** – A test that runs several real components together, such as a handler and a database. §16.1
- **Interface (TS)** – A named object type in TypeScript; can be extended and merged. §16.10
- **IntersectionObserver** – Reports when an element enters or leaves the viewport; used for lazy loading and infinite scroll. §14.9

- **`Intl`** – The internationalization namespace: `DateTimeFormat`, `NumberFormat`, `Collator`, `Segmenter` and more. §5.11
- **`Iterable`** – An object with a `[Symbol.iterator]()` method, usable in `for…of` and spread. §8.4
- **`Iterator`** – An object with a `next()` method returning `{ value, done }`. §8.4
- **`Iterator` helpers** – Lazy `map`, `filter`, `take`, `drop`, `flatMap` and consuming `toArray`, `reduce` on iterators (ES2025). §8.6

J

- **JavaScriptCore** – Apple's JavaScript engine, used by Safari and Bun. §1.3
- **JIT compilation** – Just-in-time compilation: the engine compiles frequently run code to machine code while the program runs. §1.3
- **JSDoc** – Structured comments (`/** @param {number} x */`) that document code and can drive TypeScript checking. §16.15
- **JSON** – A text data format with double-quoted keys and strings, numbers, booleans, `null`, arrays and objects. §5.12
- **JSON module** – A `.json` file imported directly with `with { type: 'json' }` (ES2025). §10.6
- **`JSON.parse`** – Turns JSON text into values; strict syntax; an optional reviver transforms values. §5.12
- **`JSON.stringify`** – Turns values into JSON text; drops functions and `undefined` in objects, throws on BigInt and cycles. §5.12

L

- **Label** – A name before a loop (`outer:`) that `break outer` / `continue outer` can target. §3.11
- **Layout thrashing** – Alternating DOM writes and layout reads so the browser recalculates layout again and again. §13.11
- **Lazy evaluation** – Computing values only when they're requested, as generators and iterator helpers do. §8.5
- **Lazy quantifier** – A regex repetition with a trailing `?` (`.+?`) that matches as little as possible. §6.7
- **`let`** – Declares a block-scoped variable that can be reassigned. §2.1
- **Lexical scope** – Scope decided by where code is written, not where it's called from. §4.5
- **Lexical `this`** – An arrow function's `this` is the `this` of the surrounding code. §4.3
- **libuv** – The C library that gives Node its event loop, async I/O and thread pool. §15.1
- **Linter** – A tool that analyses source code for bugs and style problems without running it (ESLint). §10.13
- **Live binding** – An ES module import stays connected to the exporter's variable and sees updates. §10.3
- **Live collection** – A DOM collection (`getElementsByTagName`) that updates automatically when the document changes. §13.2
- **`localStorage`** – Per-origin string key-value storage that survives reloads; synchronous, about 5 MB. §14.3
- **Lockfile** – `package-lock.json`: records the exact installed version of every dependency. §10.10
- **Logical assignment** – `a ||= b`, `a &&= b`, `a ??= b`: assign only when the condition holds (ES2021). §2.7
- **Lookahead / lookbehind** – Regex assertions `(?=…)`, `(?!…)`, `(?<=…)`, `(?<!…)` that check surrounding text without consuming it. §6.7

M

- **Macrotask** – Another name for a task in the event loop (timers, I/O, events). §12.2
- **`Map`** – A key-value collection with keys of any type, insertion order and a `size`. §8.1
- **`Math`** – A namespace object with maths constants and functions (`round`, `max`, `random`). §5.10
- **Memoization** – Caching a function's results by its arguments so repeated calls are instant. §4.7
- **Memory leak** – Memory that stays reachable by mistake (forgotten listeners, timers, growing caches, detached DOM nodes). §17.4.7
- **Microtask** – A job that runs right after the current task finishes: promise reactions, code after `await`, `queueMicrotask`. §12.2
- **Minification** – Removing whitespace and shortening names so bundles download faster. §10.11
- **Mixin** – A reusable set of methods added to unrelated classes, by copying or by a subclass factory. §9.8
- **Mock** – A test double that records calls so the test can verify how it was used. §16.5
- **Module** – A file with its own scope that exports and imports bindings. §10.1

- **Module pattern** – An IIFE that returns an object of public functions while keeping private state in a closure. §4.9
- **Mutating method** – An array method that changes the original array (`push`, `splice`, `sort`, `reverse`). §7.4
- **MutationObserver** – Calls back (as a microtask) with a batch of changes when the observed DOM changes. §13.10

N

- **Named capture group** – `(?<name>…)` in a regex; its match appears as `match.groups.name`. §6.7
- **Named export** – An export referenced by its name: `export const x`, imported with `import { x }`. §10.2
- **Namespace import** – `import * as ns from './m.mjs'`: one object holding all of a module's exports. §10.2
- `NaN` – "Not a Number": a `number` value from failed maths; not equal to anything, including itself. §2.10
- **Narrowing (TS)** – Refining a broad type to a specific one via checks like `typeof`, `in` or a discriminant. §16.11
- `new` – Creates an object linked to `Fn.prototype`, runs the constructor with it as `this` and returns it. §9.4
- `new.target` – Inside a function, tells whether it was called with `new` (and which constructor). §9.4
- `node:test` – Node's built-in test runner with suites, hooks, mocks, snapshots and coverage. §16.3
- **Node.js** – A JavaScript runtime built from V8 and libuv, for servers, CLIs and tools. §15.1
- **NodeList** – A list of DOM nodes; `querySelectorAll` returns a static one that has `forEach`. §13.2
- **Non-capturing group** – `(?:…)`: groups part of a regex without remembering the match. §6.7
- **npm** – Node's package manager and public registry. §10.9
- `npm ci` – Clean, exact install from the lockfile; the right command for CI and deploys. §10.10
- `null` – A primitive meaning "deliberately empty"; `typeof null` is `'object'`. §2.3
- **Nullish** – The two "no value" values: `null` and `undefined`. §2.7
- **Nullish coalescing** – `a ?? b`: `b` only when `a` is `null` or `undefined` (ES2020). §2.7
- `Number.EPSILON` – The gap between 1 and the next double, about 2.2e-16; a tolerance for float comparisons. §2.10
- `Number.MAX_SAFE_INTEGER` – $2^{53} - 1$, the largest integer that numbers represent exactly. §2.10
- **Numeric separator** – Underscores inside number literals for readability: `1_000_000` (ES2021). §2.10

O

- `Object.create` – Makes a new object with a chosen prototype, or none: `Object.create(null)`. §9.3
- `Object.entries` – Returns an object's own enumerable `[key, value]` pairs. §5.6
- `Object.freeze` – Makes an object's own properties read-only and blocks new ones; shallow. §5.6
- `Object.is` – Like `===` except `Object.is(NaN, NaN)` is true and `Object.is(0, -0)` is false. §2.6
- **Optional catch binding** – `try { } catch { }` without naming the error (ES2019). §11.2
- **Optional chaining** – `a?.b`, `a?.[k]`, `a?.()`: stop and return `undefined` if the left side is `null` / `undefined`. §5.8
- **Origin** – The scheme, host and port of a URL; the unit of the same-origin policy. §14.2
- **Own property** – A property stored directly on an object rather than inherited. §9.3

P

- `package.json` – A project's manifest: name, version, scripts, dependencies, `type` and entry points. §10.9
- `padStart` / `padEnd` – Pad a string to a length with a fill string (`'5'.padStart(3, '0')` → `'005'`). §6.6
- **Partial application** – Fixing some of a function's arguments now and supplying the rest later. §4.8
- **Passive listener** – A listener registered with `{ passive: true }`, promising not to call `preventDefault()`, so scrolling stays smooth. §13.8
- `path` module – Node's utilities for joining, resolving and splitting file paths. §15.5
- **Permission model** – Node's `--permission` flag that denies file-system reads and writes, child processes, worker threads and more unless each is allowed (`--allow-fs-read=…`). §15.12
- `pipeline` – Connects streams, handles backpressure and errors, and returns a promise (from `node:stream/promises`). §15.9
- **Polyfill** – Code that implements a missing built-in API for older environments. §10.12
- `popstate` – Event fired when the user moves back or forward through history entries. §14.7

- **Precedence** — The order in which operators are evaluated when parentheses don't decide it. §2.9
- **Preflight** — The `OPTIONS` request a browser sends before a non-simple cross-origin request to ask permission. §14.2
- **Prettier** — An opinionated code formatter. §10.13
- `preventDefault` — Cancels the browser's default action for an event, such as following a link or submitting a form. §13.8
- **Primitive** — An immutable value that isn't an object: number, bigint, string, boolean, undefined, null, symbol. §2.4
- **Private field** — A `#name` class member only reachable inside the class body, enforced at runtime (ES2022). §9.5
- `process` — Node's global object for arguments, environment variables, exit codes and signals. §15.6
- `process.nextTick` — Node queue that runs before promise microtasks after the current operation. §12.4
- **Promise** — An object representing a value that will be available later; pending, then fulfilled or rejected. §12.7
- `Promise.all` — Waits for all promises; values in input order; rejects on the first rejection. §12.8
- `Promise.allSettled` — Waits for every promise and reports each outcome; never rejects. §12.8
- `Promise.any` — Fulfils with the first fulfilment; rejects with `AggregateError` if all reject. §12.8
- `Promise.race` — Settles like whichever input settles first. §12.8
- **Promisify** — Converting a callback-style function into one that returns a promise (`util.promisify`). §12.7
- **Property (DOM)** — The live JavaScript value on an element object (`input.value`), as opposed to the HTML attribute. §13.4
- **Property descriptor** — The settings of a property: `value`, `writable`, `enumerable`, `configurable`, or `get` / `set`. §5.6
- **Prototype** — The object another object delegates missing property reads to, via its `[[Prototype]]` link. §9.3
- **Prototype chain** — The linked series of prototypes searched when a property isn't found on the object itself. §9.3
- **Prototype pollution** — An attack that writes to `Object.prototype` (e.g. via a `__proto__` key in merged JSON), affecting every object. §9.3
- **Proxy** — An object that wraps another and intercepts operations like get and set. §5.9
- **Pure function** — A function whose output depends only on its inputs and which has no side effects. §4.11

Q

- `querySelector` — Returns the first element matching a CSS selector, or `null`. §13.2
- `queueMicrotask` — Schedules a function as a microtask. §12.5

R

- **Realm** — A separate global environment (an iframe, a `vm` context) with its own copies of built-ins. §9.7
- **Recursion** — A function solving a problem by calling itself on a smaller version of it. §4.10
- `reduce` — Folds an array into one value by running an accumulator function; pass an initial value. §7.5
- **Reflow** — The browser recalculating element positions and sizes (layout) after DOM or style changes. §13.11
- `RegExp.escape` — Escapes special regex characters so a string can be matched literally (ES2025). §6.9
- **Regular expression** — A pattern for matching text, written `/pattern/flags` or with `new RegExp`. §6.7
- **Rejection** — A promise settling with an error (reason) instead of a value. §12.7
- **Remainder operator** — `%`: the remainder of division, with the sign of the left operand. §2.5
- **Rendering (event loop)** — The step where the browser runs `requestAnimationFrame` callbacks, then style, layout and paint. §12.2
- **Repaint** — Redrawing pixels after a visual change that doesn't affect layout. §13.11
- `replaceAll` — Replaces every occurrence of a string or global regex (ES2021). §6.6
- `requestAnimationFrame` — Runs a callback right before the next paint; the right tool for animations. §14.5
- `ResizeObserver` — Reports when an element's size changes. §14.9
- **Rest parameter** — `...args` as the last parameter: collects remaining arguments into a real array. §4.4
- **Reviver** — A function passed to `JSON.parse` to transform each parsed value. §5.12

- **Run-to-completion** – Once a task starts, it runs until the call stack is empty; nothing interrupts it. §12.1

S

- **Same-origin policy** – Browser rule that a page may only read responses from its own origin unless CORS allows it. §14.2
- `SameSite` – Cookie attribute (`Strict`, `Lax`, `None`) controlling whether it's sent on cross-site requests. §14.11
- `satisfies` – TypeScript operator that checks a value against a type without widening the value's own type. §16.13
- **Scope** – The region of code where a name is visible: global, module, function or block. §4.5
- **Scope chain** – The nested scopes searched outward when a variable is looked up. §4.5
- **Seal** – `Object.seal`: no properties can be added or removed, but values can change. §5.6
- **Semantic versioning** – `MAJOR.MINOR.PATCH`: breaking changes, new features, bug fixes. §10.10
- **Service worker** – A background script that acts as a network proxy for offline support, caching and push. §14.8
- `sessionStorage` – Like `localStorage` but scoped to one tab and cleared when it closes. §14.3
- `Set` – A collection of unique values in insertion order with O(1) average `has`. §8.2
- **Set methods** – `union`, `intersection`, `difference`, `symmetricDifference` and subset tests on Sets (ES2025). §8.2
- `setImmediate` – Node timer that runs in the check phase, after I/O callbacks. §12.4
- `setInterval` – Runs a function repeatedly roughly every N milliseconds until cleared. §12.5
- **Setter** – A `set` method that runs when a property is assigned. §5.7
- `setTimeout` – Runs a function once after at least N milliseconds, as a new task. §12.5
- **Shadowing** – An inner variable or own property hiding one with the same name further out. §4.5
- **Shallow copy** – A copy of the top level only; nested objects are still shared. §5.4
- **Short-circuit evaluation** – `&&`, `||`, `??` and `?.` stop evaluating as soon as the result is known. §2.7
- **Side effect** – Anything a function does beyond returning a value: mutating inputs, I/O, logging, changing globals. §4.11
- **Single-threaded** – JavaScript runs one piece of your code at a time on one thread; waiting is handed to the host. §12.1
- **Snapshot test** – A test that saves a serialized output and fails when it changes. §16.6
- **Source map** – A file mapping minified or transpiled code back to the original source for debugging. §11.8
- `splice` – Mutating array method that removes and/or inserts items at an index and returns the removed items. §7.4
- **Spread syntax** – `...` expanding an iterable into elements or arguments, or an object's properties into another object. §5.4
- **Spy** – A test double that records its calls (`t.mock.fn()`). §16.5
- **Stack trace** – The list of function calls active when an error was created, newest first. §11.8
- **Stage (TC39)** – A proposal's maturity from 0 (idea) to 4 (finished, goes into the next edition). §1.2
- **Statement** – A complete instruction such as a declaration, an `if` or a loop. §1.6
- **Static initialization block** – `static { … }` inside a class: code that runs once when the class is defined (ES2022). §9.5
- **Static member** – A method or field on the class itself, not on instances: `static create()`. §9.5
- `stopPropagation` – Stops an event from travelling to further elements. §13.6
- **Stream** – Node's abstraction for data processed in chunks: Readable, Writable, Duplex, Transform. §15.9
- **Strict equality** – `===`: same type and value, no conversion. §2.6
- **Strict mode** – A restricted mode that turns silent mistakes into errors; automatic in modules and classes. §1.8
- **String** – An immutable sequence of UTF-16 code units. §6.1
- `structuredClone` – Built-in deep copy that handles Dates, Maps, Sets and cycles, but not functions. §5.4
- **Stub** – A test double that returns canned answers. §16.5
- **Subresource Integrity** – An `integrity` hash on a `<script>` so the browser refuses a tampered CDN file. §14.12

- `super` – Calls the parent class's constructor (`super()`) or methods (`super.m()`). §9.6
- `switch` – Compares one value against `case` values with `===`; falls through without `break`. §3.4
- **Symbol** – A primitive creating unique identifiers, often used as property keys. §2.3
- `Symbol.iterator` – The well-known symbol naming the method that makes an object iterable. §8.4
- **Synchronous** – Code that runs step by step and blocks until each step finishes. §12.1

T

- **Tagged template** – A function called with a template literal's pieces: `tag`Hi ${name}``. §6.3
- **Task** – A unit of work in the event loop: a script, timer callback, I/O callback or UI event. §12.2
- **TC39** – The Ecma committee that designs and maintains ECMAScript. §1.2
- **Template literal** – A backtick string with `${}` interpolation and real line breaks. §6.3
- **Temporal** – The modern date-time API (Stage 4 in March 2026, planned for ES2027); not in Node 24. §5.11
- **Temporal Dead Zone (TDZ)** – The time between entering a scope and a `let`/`const`/`class` declaration running; access throws. §2.2
- **Ternary operator** – `cond ? a : b`: chooses between two values. §3.3
- `textContent` – All text inside a node; setting it inserts plain text, never HTML. §13.3
- `then` – Registers callbacks on a promise and returns a new promise. §12.7
- `this` – The context object of a function call, set by how the function is called. §9.1
- **Thread pool** – libuv's worker threads (4 by default) that run file-system, DNS lookup, crypto and zlib jobs for Node. §15.2
- **Throttle** – Run a function at most once per interval during a burst of calls (e.g. scroll handlers). §17.3
- `throw` – Raises an exception, stopping normal flow until a `catch` handles it. §11.2
- **Top-level** `await` – Using `await` at the top level of an ES module; importers wait for it (ES2022). §10.5
- `toSorted` / `toReversed` / `toSpliced` / `with` – Non-mutating copies of `sort`, `reverse`, `splice` and index assignment (ES2023). §7.4
- **Transpiler** – A tool that rewrites source code into another syntax (TypeScript or new JavaScript → older JavaScript). §10.12
- **Tree shaking** – Removing unused exports from a bundle, made possible by static ES module syntax. §10.11
- **Truthy** – Any value that is not falsy, including `[]`, `{}` and `'0'`. §2.11
- `try…catch` – Runs code and handles exceptions thrown synchronously inside it. §11.2
- **Tuple (TS)** – A fixed-length array type with a type per position, e.g. `[string, number]`. §16.10
- **Type annotation** – A TypeScript declaration of a value's type, such as `x: number`. §16.10
- **Type predicate** – A TypeScript return type `x is T` that narrows the argument when the function returns true. §16.11
- **Type stripping** – Node running `.ts` files by deleting type syntax without checking it. §16.9
- **Typed array** – An array-like view of fixed-size numbers over an `ArrayBuffer`, such as `Uint8Array`. §7.9
- `typeof` – Operator returning a string naming a value's type; `typeof null` is `'object'`. §2.3
- **TypeScript** – JavaScript plus static types that are checked by `tsc` and erased before running. §16.8

U

- `undefined` – A primitive meaning "no value assigned yet"; what missing properties and arguments give. §2.3
- **Unhandled rejection** – A rejected promise with no handler; reported by the browser and fatal by default in Node. §11.5
- **Unicode property escape** – `\p{…}` in a `u`/`v` regex, matching characters by Unicode property such as `\p{L}`. §6.9
- **Unit test** – A fast test of one function or module in isolation. §16.1
- `unknown` **(TS)** – The type-safe "anything" type: you must narrow it before using it. §16.10
- **URL / URLSearchParams** – Built-ins for parsing, building and encoding URLs and query strings. §14.6
- `using` – Declaration that automatically calls `[Symbol.dispose]()` when the block ends (finished proposal, ES2027; works in Node 24). §11.6
- **UTF-16** – The encoding JavaScript strings use: characters as one or two 16-bit code units. §6.5

V

- **V8** – Google's JavaScript engine, used in Chrome, Edge, Node.js and Deno. §1.3

- `var` – The original function-scoped, hoisted variable declaration; avoid in new code. §2.1
- **Vite** – A dev server plus production bundler; the default choice for new front-end apps. §10.11
- **Void operator** – `void expr` evaluates the expression and returns `undefined`. §2.9

W

- **WeakMap** – A map with object keys held weakly: entries disappear when the key object is garbage-collected. §8.3
- **WeakRef** – A weak reference to an object; `deref()` returns it or `undefined` once collected. §8.3
- **WeakSet** – A set of objects held weakly; used to tag objects without keeping them alive. §8.3
- **Web Worker** – A background thread with no DOM access that talks to the page via `postMessage`. §14.8
- **webpack** – A long-established, highly configurable bundler. §10.11
- **Worker threads** – Node's `worker_threads` module for running CPU-heavy JavaScript in parallel. §15.2

X

- **XSS** – Cross-site scripting: attacker-controlled script runs in your page, usually through unescaped HTML. §14.10

Y

- `yield` – Pauses a generator and hands a value to the consumer; `yield*` delegates to another iterable. §8.5

Z

- **Zalgo** – A callback API that is sometimes synchronous and sometimes asynchronous, making ordering unpredictable. §12.6

Chapter 18 TL;DR

- Use this chapter for **revision**: cover the answer column of a sheet, say the answer, then follow the § link for anything you missed.
- **Language core**: `const` by default, `===` always, `??` for defaults, eight types with `typeof null === 'object'`, eight falsy values, floats that aren't exact (`0.1 + 0.2`), closures that remember their scope, and `this` decided at the call site (arrows excepted).
- **Data**: shallow spread vs `structuredClone`; `Object.freeze` is shallow; `toSorted` / `with` instead of mutating shared arrays; `Map` / `Set` for keyed lookup and uniqueness; generators and iterator helpers for lazy sequences.
- **Async**: one task, then **all** microtasks, then render; sync code → microtasks → tasks; `Promise.all` for parallel work, `allSettled` for every outcome, `AbortController` to cancel.
- **Platform**: `textContent` over `innerHTML`, delegation with `closest`, check `res.ok` after `fetch`, `HttpOnly; Secure; SameSite` cookies for sessions, CSP against XSS; in Node, `node:` modules, streams with `pipeline`, never block the loop.
- **Quality**: `node:test` + `assert/strict`, `tsc --noEmit` for types, ESLint + Prettier, lockfile + `npm ci`.
- **Versions**: ES2015 was the big one; since then one edition every June. ES2026 is only partly in Node 24; Temporal and `using` belong to ES2027.

This is the end of the JavaScript Handbook. Next: the React Handbook, or back to Chapter 1 for a second pass.